

Apostila de Preparação para Programação **C**

Rafael Coelho

14 de setembro de 2026

Prefácio

A linguagem C ocupa um papel fundamental na formação de profissionais da Computação. Embora linguagens modernas ofereçam mecanismos que abstraem diversos detalhes da implementação, compreender como um programa é executado, como os dados são organizados na memória e como os recursos do computador são utilizados continua sendo uma habilidade indispensável para quem deseja construir uma base sólida em programação.

Esta apostila foi elaborada como material de apoio às monitorias da disciplina de Linguagem C. Seu propósito não é substituir as aulas ou os livros clássicos da área, mas complementar o processo de aprendizagem por meio de explicações objetivas, exemplos comentados, diagramas de memória, exercícios progressivos e observações sobre os erros mais frequentes encontrados pelos estudantes.

Ao longo dos capítulos, a linguagem será apresentada não apenas como uma ferramenta para escrever código, mas como um meio para compreender os fundamentos da programação e o funcionamento interno dos computadores. Sempre que possível, os conceitos serão relacionados a situações práticas e acompanhados de representações visuais que auxiliem na construção de um modelo mental consistente sobre a execução dos programas.

Espera-se que este material sirva como um roteiro de estudos para alunos que desejam consolidar os conhecimentos adquiridos em sala de aula, preparar-se para avaliações e, principalmente, desenvolver uma compreensão mais profunda dos princípios que fundamentam a programação em baixo nível.

Bom estudo!

Como utilizar esta apostila

Esta apostila foi organizada para funcionar como um roteiro de aprendizagem. Em vez de apresentar apenas conceitos e exemplos, cada capítulo segue uma estrutura didática pensada para conduzir o estudante desde a motivação inicial até a aplicação prática do conteúdo.

Cada capítulo inicia com uma breve introdução, contextualizando a importância do tema e sua relação com os assuntos já estudados. Em seguida, o desenvolvimento apresenta os conceitos fundamentais, exemplos comentados e trechos de código que ilustram sua aplicação prática. Sempre que pertinente, caixas de observação, curiosidades e erros comuns destacam informações importantes, aplicações interessantes e equívocos frequentes entre estudantes, especialmente aqueles que estão migrando de linguagens de alto nível, como Python, para a linguagem C.

Ao final de cada capítulo, um resumo sintetiza os principais conceitos estudados, seguido por uma lista de objetivos de aprendizagem que funciona como um checklist para revisão. Por fim, são propostos três exercícios em ordem crescente de dificuldade: o primeiro busca consolidar os conceitos apresentados; o segundo incentiva sua aplicação na resolução de pequenos problemas; e o terceiro propõe um desafio ou reflexão que prepara o terreno para os capítulos seguintes.

Recomenda-se que o leitor acompanhe os exemplos, implemente os programas apresentados e tente resolver os exercícios antes de consultar soluções ou materiais complementares. A aprendizagem da programação ocorre principalmente por meio da prática, da experimentação e da análise dos próprios erros. Esta apostila foi concebida para servir como um guia ao longo desse processo, podendo ser utilizada tanto no estudo individual quanto como material de apoio às monitorias e às disciplinas introdutórias de programação em C.

Sumário

Prefácio	1
Como utilizar esta apostila	2
I Fundamentos de C	7
1 Introdução	8
1.1 O que significa programar?	9
1.2 Do problema ao algoritmo	9
1.3 Da linguagem humana à linguagem de máquina	10
1.4 O surgimento da linguagem C	11
1.5 Por que estudar C atualmente?	11
1.6 Estrutura de um programa em C	12
1.7 Compilando o primeiro programa	14
2 Variáveis, Tipos e Operadores	18
2.1 Variáveis e Declaração	18
2.2 Tipos de Dados Fundamentais	19
2.3 Representação dos Dados na Memória	20
2.4 Constantes e Modificadores	21
2.5 Operadores	21
2.5.1 Operadores Aritméticos	21
2.5.2 Operadores Relacionais	22
2.5.3 Operadores Lógicos	22

2.5.4	Operadores de Atribuição	22
2.5.5	Incremento e Decremento	22
2.6	Conversão de Tipos	23
2.7	Entrada e Saída de Dados	23
2.8	Escopo e Tempo de Vida	24
3	Controle de Fluxo	27
3.1	Condicionais	27
3.1.1	A estrutura <code>if</code>	27
3.1.2	<code>if-else</code>	28
3.1.3	Encadeando condições com <code>else if</code>	28
3.1.4	Aninhamento e chaves opcionais	29
3.1.5	O operador ternário	30
3.1.6	A estrutura <code>switch</code>	30
3.2	Laços	31
3.2.1	O laço <code>while</code>	31
3.2.2	O laço <code>do-while</code>	32
3.2.3	O laço <code>for</code>	32
3.2.4	<code>break</code> e <code>continue</code>	33
3.2.5	Laços aninhados	34
4	Funções	37
4.1	Declaração	38
4.1.1	Funções sem retorno: <code>void</code>	39
4.1.2	Funções sem parâmetros	39
4.2	Protótipos	40
4.3	Passagem por Valor	41
II	Organização dos Dados	46
5	Vetores	47

5.1	Conceito de Vetor	47
5.2	Declaração e Inicialização	48
5.3	Acesso aos Elementos	48
5.4	Percorrendo Vetores	49
5.5	Vetores e Funções	50
5.6	Vetores e Ponteiros	51
5.7	Operações Clássicas sobre Vetores	52
5.8	Erros Comuns	53
6	Strings	56
6.1	O que é uma String em C?	56
6.2	Inicialização de Strings	56
6.3	O Caractere '\0'	57
6.4	Entrada e Saída de Strings	58
6.5	Funções da Biblioteca <string.h>	58
6.6	Strings e Funções	59
6.7	Percorrendo Strings Manualmente	60
6.8	Erros Comuns	61
7	Arrays Multidimensionais	64
7.1	Conceito de Arrays Multidimensionais	64
7.2	Declaração e Inicialização	65
7.3	Acesso aos Elementos	66
7.4	Percorrendo Matrizes	66
7.5	Matrizes e Memória	67
7.6	Matrizes e Funções	67
7.7	Operações Clássicas	68
7.8	Matrizes como Aplicações	69
7.9	Erros Comuns	69

III	Gerenciamento de Memória	72
8	Ponteiros	73
9	Organização da Memória	74
9.1	Segmento de Código	74
9.2	Stack	74
9.3	Heap	74
9.4	Data	74
9.5	BSS	74
10	Alocação Dinâmica	75
IV	Programação Modular	76
11	Structs e Enums	77
12	Modularização	78
12.1	Arquivos Header	78
12.2	Include Guards	78
12.3	Compilação Separada	78
12.4	Linkagem	78
13	Estruturas de Dados Básicas	79
V	Tópicos Avançados	80
14	Manipulação Bit a Bit	81
15	Recursão	82
16	Depuração	83
VI	Para ir além	84

Parte I

Fundamentos de C

1 Introdução

Programar é, antes de tudo, resolver problemas. Em sua essência, um programa de computador nada mais é do que uma sequência organizada de instruções capaz de transformar dados de entrada em resultados úteis. Porém, essa definição simples esconde um aspecto fundamental: computadores não compreendem intenções, ideias ou objetivos. Eles executam apenas instruções elementares, em uma ordem rigorosamente definida, realizando milhões ou até bilhões de operações por segundo.

Ao longo da história da Computação, diferentes linguagens de programação foram desenvolvidas para reduzir a distância entre a forma como os seres humanos pensam e a forma como os computadores executam instruções. Linguagens de alto nível, como Python, permitem que problemas complexos sejam descritos por meio de comandos simples e expressivos, abstraindo detalhes relacionados à organização da memória, ao gerenciamento de recursos e à arquitetura do processador. Essa capacidade torna o desenvolvimento de software mais produtivo e acessível, permitindo que o programador concentre seus esforços na lógica da solução.

Entretanto, compreender apenas essas abstrações é conhecer apenas uma parte do processo. Em diversas áreas da Computação, como sistemas operacionais, compiladores, sistemas embarcados, arquitetura de computadores, processamento de alto desempenho e desenvolvimento de drivers, torna-se necessário compreender como os dados são realmente armazenados, manipulados e transferidos pela máquina. É nesse contexto que a linguagem C assume um papel de destaque.

Desenvolvida no início da década de 1970 por Dennis Ritchie, nos Laboratórios Bell, a linguagem C surgiu com o objetivo de oferecer uma alternativa eficiente e portátil para o desenvolvimento do sistema operacional UNIX. Desde então, tornou-se uma das linguagens mais influentes da história da Computação, servindo de base para inúmeras outras linguagens modernas e permanecendo amplamente utilizada em aplicações nas quais desempenho, controle de memória e acesso direto ao *hardware* são requisitos fundamentais.

Estudar C não significa abandonar as facilidades oferecidas pelas linguagens mo-

dermas, mas compreender os mecanismos que tornam essas facilidades possíveis. Conceitos como ponteiros, organização da memória, pilha de execução, alocação dinâmica e representação binária dos dados constituem fundamentos presentes, de forma direta ou indireta, em praticamente todas as áreas da Computação. Assim, aprender C representa uma oportunidade de desenvolver uma compreensão mais profunda sobre o funcionamento dos programas e dos computadores.

Ao longo desta apostila, os conceitos serão apresentados de maneira gradual, sempre buscando estabelecer uma relação entre a escrita do código e o comportamento do programa durante sua execução. Sempre que possível, serão utilizados diagramas de memória, exemplos comentados e exercícios progressivos para auxiliar na construção de um modelo mental consistente sobre o funcionamento interno da linguagem.

1.1 O que significa programar?

Programar é, antes de tudo, uma forma de resolver problemas. Quando escrevemos um programa, estamos descrevendo, passo a passo, como um computador deve agir para transformar uma entrada em uma saída desejada. Não é sobre "escrever código" no sentido mecânico — é sobre pensar de forma estruturada, quebrar um problema grande em partes menores e traduzir esse raciocínio em instruções que uma máquina consiga executar sem ambiguidade.

Diferente de um ser humano, o computador não interpreta intenções. Ele não "entende" o que você quer dizer, apenas o que você efetivamente escreveu. Por isso, programar exige precisão: cada instrução precisa ser exata, cada condição precisa estar bem definida, cada passo precisa ter uma ordem clara. Essa exigência de rigor é, ao mesmo tempo, o que torna a programação desafiadora no início e extremamente poderosa depois que você domina o processo — porque uma vez que o computador sabe exatamente o que fazer, ele faz isso rapidamente, repetidamente e sem erros de execução.

Aprender a programar, portanto, é menos sobre decorar sintaxe e mais sobre desenvolver uma forma de pensar: o chamado **pensamento computacional**. É a habilidade de olhar para um problema do mundo real e enxergar nele uma sequência lógica de passos que pode ser automatizada.

1.2 Do problema ao algoritmo

Antes de qualquer linha de código, existe um problema. E antes de qualquer programa, existe um **algoritmo**: uma sequência finita e bem definida de passos que resolve esse

problema.

Pense em uma tarefa simples do dia a dia, como fazer um café. Existe uma ordem lógica de ações: ferver a água, moer o pó (se necessário), colocar o pó no filtro, despejar a água, esperar coar. Se você trocar a ordem dos passos ou pular algum, o resultado muda ou simplesmente não funciona. Um algoritmo de programação segue a mesma lógica, só que aplicado a problemas computacionais: somar dois números, ordenar uma lista de nomes, verificar se uma senha é válida, calcular a média de notas de uma turma.

O processo de desenvolvimento de software segue, de forma geral, este caminho:

1. **Entender o problema:** o que exatamente precisa ser resolvido?
2. **Projetar o algoritmo:** quais passos, em qual ordem, resolvem esse problema?
3. **Implementá-lo:** traduzir o algoritmo para uma linguagem de programação.
4. **Testar:** verificar se o programa realmente resolve o problema em diferentes situações.

É comum que iniciantes pulem direto para o passo 3, tentando escrever código antes de entender o problema ou pensar no algoritmo. Isso costuma gerar programas confusos e cheios de erros. Uma boa prática que vamos reforçar ao longo de toda a apostila é sempre rascunhar o algoritmo antes de programar, seja em português mesmo, seja em pseudocódigo, seja em um simples diagrama.

1.3 Da linguagem humana à linguagem de máquina

Nós pensamos e nos comunicamos em linguagem natural: português, inglês, e assim por diante. O computador, por outro lado, só entende sequências de 0s e 1s a chamada **linguagem de máquina**. Entre esses dois extremos existe uma escada de abstração, e é nela que as linguagens de programação se encaixam.

- No nível mais baixo está o **código binário**, que o processador executa diretamente.
- Um pouco acima está o **assembly**, que usa mnemônicos (como ‘MOV’, ‘ADD’, ‘JMP’) para representar instruções de máquina de forma um pouco mais legível para humanos, mas ainda extremamente ligada ao hardware.
- Acima disso estão as **linguagens de alto nível**, como C, Python, Java, entre outras, que usam palavras e estruturas mais próximas do raciocínio humano.

Quando você escreve um programa em C, esse código não é executado diretamente pelo processador. Ele precisa passar por um **compilador**, um programa que traduz o código-fonte escrito por você para linguagem de máquina, gerando um arquivo executável.

C ocupa uma posição interessante nessa escada: é considerada uma linguagem de **médio nível**. Ela tem estruturas legíveis para humanos (laços, condicionais, funções), mas também permite manipular a memória do computador de forma bastante direta algo que linguagens mais modernas costumam esconder do programador. É justamente esse equilíbrio que torna C tão importante de se entender, como veremos a seguir.

1.4 O surgimento da linguagem C

A linguagem C foi criada no início da década de 1970 por **Dennis Ritchie**, nos Laboratórios Bell (Bell Labs), com o objetivo original de desenvolver o sistema operacional **Unix**. Antes de C, sistemas operacionais eram escritos quase inteiramente em assembly um processo lento, difícil de manter e completamente dependente do hardware específico de cada máquina.

C nasceu como evolução de linguagens anteriores, como a **B** (criada por Ken Thompson) e a **BCPL**. A grande sacada de Ritchie foi criar uma linguagem poderosa o suficiente para escrever um sistema operacional inteiro, mas portátil o bastante para rodar em diferentes arquiteturas de hardware sem precisar reescrever tudo do zero.

O sucesso foi tão grande que, em pouco tempo, o próprio Unix foi reescrito em C, e a linguagem se espalhou junto com a adoção do sistema pelo mundo acadêmico e, depois, comercial. Em 1978, Brian Kernighan e Dennis Ritchie publicaram o livro *The C Programming Language*, obra que se tornou referência tão importante que muita gente ainda chama a versão clássica da linguagem de "C K&R" (Kernighan & Ritchie).

Com o tempo, C passou por processos de padronização formal ANSI C (1989), depois ISO C (C90, C99, C11, C17, C23) garantindo que compiladores diferentes, em sistemas diferentes, interpretassem a linguagem da mesma forma.

1.5 Por que estudar C atualmente?

Se você já programou em Python, provavelmente nunca precisou se preocupar com onde uma variável estava armazenada na memória. Em C, essa preocupação deixa de ser um detalhe e passa a fazer parte da programação. Então, mais de cinquenta anos depois de criada, por qual motivo C continua extremamente relevante? Não é apenas

por tradição ou nostalgia, existem razões práticas concretas:

- **C é a base de grande parte da computação moderna:** Sistemas operacionais, compiladores, bibliotecas e diversas linguagens de programação possuem componentes implementados em C. Compreender a linguagem significa compreender parte dos fundamentos sobre os quais a computação atual foi construída.
- **C permite compreender o funcionamento interno do computador:** Ao programar em C, passamos a lidar explicitamente com memória, ponteiros, organização dos dados e outros conceitos que normalmente permanecem ocultos em linguagens de alto nível. Essa experiência desenvolve uma compreensão mais profunda sobre como os programas realmente são executados.
- **C continua essencial em aplicações de baixo nível:** Sistemas embarcados, firmware, drivers de dispositivos, sistemas operacionais e diversas aplicações que exigem controle direto do hardware ainda utilizam C como uma das principais linguagens de desenvolvimento.

Em resumo: você não está aprendendo C apenas para "escrever programas em C". Está aprendendo os fundamentos que sustentam praticamente toda a computação moderna.

1.6 Estrutura de um programa em C

Tradicionalmente, o primeiro programa apresentado em uma linguagem de programação é aquele que exibe uma simples mensagem na tela. Embora pareça um exemplo modesto, esse programa reúne os principais elementos estruturais que aparecerão em praticamente todos os programas escritos em C. Por esse motivo, ele será utilizado como ponto de partida para compreender a organização básica de um programa.

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("Hello, \uworld!\n");
6     return 0;
7 }
```

Vamos analisar, a seguir, cada parte desse programa.

Curiosidade

Embora hoje seja tradição iniciar o estudo de uma linguagem de programação com um programa que imprime “Hello, World!”, essa expressão foi popularizada por Brian Kernighan. Ela apareceu inicialmente em um exemplo do livro *A Tutorial Introduction to the Language B* (1973) e, posteriormente, tornou-se mundialmente conhecida por meio do clássico *The C Programming Language* (1978), escrito por Brian Kernighan e Dennis Ritchie.

Usando a numeração das linhas temos em ordem crescente:

```
#include <stdio.h>
```

Essa linha é uma **diretiva de pré-processador**. Antes mesmo da compilação, ela instrui o pré-processador a incluir o conteúdo da biblioteca padrão de entrada e saída (`'stdio.h'`, de **standard input/output**), que contém a declaração de funções como `'printf'` (usada para exibir texto na tela) e `'scanf'` (usada para ler dados digitados pelo usuário).

```
int main(void)
```

Toda execução de um programa em C começa em uma função chamada `'main'`. É o ponto de entrada de todo programa em C. O `'int'` antes do nome indica que essa função devolve um número inteiro ao sistema operacional quando termina normalmente usado para indicar se o programa foi executado com sucesso ou não. O `'void'` entre parênteses indica que essa função não recebe parâmetros de entrada.

```
{...}
```

As chaves delimitam o corpo da função, isto é, o conjunto de instruções que será executado quando a função for chamada. No caso da função `main`, esse bloco corresponde ao código executado quando o programa é iniciado.

```
printf("Hello, world!\n");
```

A função `printf` é utilizada para exibir informações na tela. Nesse exemplo, ela recebe como argumento a sequência de caracteres `"Hello, world!\n"`, passando como argumento o texto que queremos exibir. O `'\n'` no final é um caractere especial que representa uma quebra de linha. Note o ponto e vírgula (`';'`) ao final em C, ele marca o fim de cada instrução, e sua ausência é um dos erros mais comuns entre iniciantes.

```
return 0;
```

Essa linha finaliza a função `'main'`, devolvendo o valor `'0'` ao sistema operacional. Por convenção, `'0'` significa que o programa terminou sem erros. Em programas simples, essa linha costuma aparecer no final da função `main`. Ao longo da apostila veremos que outras funções também podem utilizar a instrução `return` para devolver resultados

ao programa.

De forma geral, todo programa em C segue este esqueleto:

```
1 #include <biblioteca_1>
2 #include <biblioteca_2>
3 ...
4
5 int main(void) {
6     // instruções do programa
7     return 0;
8 }
```

Naturalmente, programas reais podem conter diversas outras funções, declarações, estruturas de dados e bibliotecas. Entretanto, a estrutura apresentada acima representa o núcleo de um programa em C e servirá como base para os próximos capítulos.

Observação

Neste momento, não se preocupe em memorizar todos os detalhes do programa. Ao longo dos próximos capítulos, cada um desses elementos será estudado individualmente. O objetivo desta seção é apenas apresentar a estrutura geral de um programa em C e familiarizar o leitor com sua organização.

1.7 Compilando o primeiro programa

Diferente de linguagens interpretadas, um programa em C precisa ser compilado antes de ser executado. O compilador lê o código-fonte (o arquivo ‘c’ que você escreveu) e gera um arquivo executável, específico para o sistema operacional e a arquitetura de hardware em que foi compilado.

Um dos compiladores mais usados, especialmente em ambientes Linux e no ensino, é o **GCC** (GNU Compiler Collection). Supondo que você tenha salvo o programa da seção 1.6 em um arquivo chamado ‘ola_mundo.c’, o processo básico de compilação e execução no terminal seria:

```
1 aluno@ubuntu:~$ gcc ola_mundo.c -o ola_mundo
2
3 aluno@ubuntu:~$ ./ola_mundo
```

Explicando cada parte:

<code>gcc</code>	Invoca o compilador GCC.
<code>ola_mundo.c</code>	Arquivo-fonte que será compilado.
<code>-o ola_mundo</code>	Define o nome do executável gerado.
<code>./ola_mundo</code>	Executa o programa localizado no diretório atual.

Assim, se tudo estiver correto, a saída no terminal será:

```
1 Hello, world!
```

Caso exista algum erro de sintaxe como esquecer um ponto e vírgula ou uma chave o compilador vai recusar a compilação e exibir mensagens de erro apontando a linha e o tipo de problema encontrado. Aprender a ler essas mensagens de erro é uma habilidade tão importante quanto aprender a escrever o código em si, e vamos treinar isso bastante ao longo da apostila.

Resumo do Capítulo

Parabéns! Você acabou de escrever, compilar e executar seu primeiro programa em C. Embora simples, esse exemplo percorreu todas as etapas fundamentais do desenvolvimento: partiu de um arquivo de código-fonte, passou pelo processo de compilação e resultou em um programa executável.

Ao longo do capítulo, você também conheceu o contexto histórico da linguagem C e refletiu sobre o verdadeiro papel da programação: utilizar o computador como ferramenta para resolver problemas por meio de algoritmos.

Nos próximos capítulos, ampliaremos gradualmente essa estrutura, introduzindo variáveis, operadores, estruturas de controle e funções, até que você seja capaz de desenvolver programas cada vez mais completos.

Ao final deste capítulo, você deve ser capaz de:

- Explicar, em linhas gerais, como surgiu a linguagem C e sua importância histórica.
- Diferenciar algoritmo de programa.
- Identificar os principais componentes da estrutura de um programa em C.
- Compreender o papel das diretivas de pré-processador e da função `main`.
- Compilar e executar um programa simples utilizando o GCC.
- Interpretar mensagens básicas de erro emitidas pelo compilador.

Erros comuns

Erro comum

Esquecer o ponto e vírgula (;)

Para quem teve o primeiro contato com programação em Python, é recorrente esquecer o ponto e vírgula ao final das instruções. Em C, praticamente toda instrução deve ser finalizada com `;`. Quando isso não acontece, o compilador informa um erro de sintaxe.

Python

```
1 print("Betas usam Python.")
```

C

```
1 printf("Trivial.");
```

Exercícios

Exercício

Fixação

Explique, com suas palavras, por que a linguagem C continua sendo importante para a formação de um profissional de Computação, mesmo existindo linguagens

mais modernas.

Aplicação

Escreva um programa em C que exiba seu nome, idade e cidade que nasceu na tela (com a exata formatação a seguir):

```
1 Rafael, 21
2 Aracaju-SE.
```

Em seguida, compile e execute o programa utilizando o GCC.

Reflexão

Neste capítulo, vimos que um programa em C precisa ser compilado antes de ser executado. Em Python, normalmente basta executar o arquivo diretamente.

Pesquise, em poucas linhas, qual é a diferença entre uma linguagem compilada e uma linguagem interpretada. Em sua opinião, quais vantagens e desvantagens cada abordagem apresenta?

2 Variáveis, Tipos e Operadores

Todo programa existe para representar informações e operar sobre elas. Se você já teve contato com Python, provavelmente está acostumado a pensar em uma variável simplesmente como "uma caixa que guarda um valor" você escreve `idade = 20` e pronto, o Python descobre sozinho que aquilo é um número inteiro.

Em C, esse raciocínio precisa ser mais preciso. Antes de guardar qualquer valor, precisamos responder três perguntas:

- Qual é o tipo desse dado?
- Quanto espaço ele ocupa na memória?
- Como esse valor é efetivamente representado, em bits, dentro desse espaço?

Essas perguntas podem parecer burocráticas no início, mas são exatamente o que torna C uma linguagem tão próxima do funcionamento real do computador. Neste capítulo, vamos construir a base para pensar em variáveis não como "caixas mágicas", mas como regiões de memória com um tipo bem definido.

2.1 Variáveis e Declaração

Uma variável é um espaço nomeado na memória do computador, usado para armazenar um valor que pode ser lido e modificado durante a execução do programa. Em C, toda variável precisa ser **declarada** antes de ser usada ou seja, você precisa informar ao compilador qual é o tipo dela e qual nome ela vai ter.

```
1 int idade;  
2 idade = 20;  
3  
4 int ano = 2026;
```

Note que existem duas ações diferentes acontecendo:

- **Declaração:** `int idade;` aqui você está dizendo ao compilador "reserve espaço na memória para um número inteiro chamado idade".
- **Atribuição:** `idade = 20;` aqui você está colocando o valor 20 nesse espaço já reservado.

Quando as duas coisas acontecem na mesma linha, como em `int ano = 2026;`, chamamos isso de inicialização — declaração e atribuição em um único passo.

Regras para identificadores (nomes de variáveis) em C:

1. Podem conter letras, dígitos e underscore (`_`).
2. Não podem começar com um dígito.
3. São case-sensitive (`idade` e `Idade` são variáveis diferentes).
4. Não podem ser palavras reservadas da linguagem (como `int`, `return`, `if`).

A diferença em relação ao Python: em Python, uma mesma variável pode receber valores de tipos diferentes ao longo do programa (`x = 10` e depois `x = "texto"` são ambos válidos). Em C isso não é possível — uma vez declarada com um tipo, a variável só pode armazenar valores compatíveis com aquele tipo durante toda a sua existência.

Erro comum

Declarar uma variável local e tentar usá-la antes de atribuir um valor:

```
1 int total;  
2 printf("%d", total); // comportamento indefinido!
```

Diferente de variáveis globais (que começam zeradas automaticamente), variáveis locais não têm um valor inicial garantido. Elas contêm "lixo de memória" o que quer que estivesse naquele espaço antes.

Sempre inicialize suas variáveis antes de usá-las.

2.2 Tipos de Dados Fundamentais

C possui alguns tipos básicos, chamados de **tipos primitivos**, que servem de base para tudo o que vamos construir daqui em diante:

Tipo	Representa	Exemplo
char	um único caractere	char letra = 'A';
int	um número inteiro	int quantidade = 42;
float	número real (precisão simples)	float altura = 1.75f;
double	número real (precisão dupla)	double pi = 3.14159265;
void	ausência de tipo/valor	usado principalmente em funções

Mais adiante, vamos discutir sobre o tipo `bool`, que representa valores verdadeiro/falso, em C com mais calma quando chegarmos às estruturas condicionais.

Por enquanto, não vamos nos aprofundar em modificadores como `short`, `long`, `signed` e `unsigned` eles merecem atenção própria e voltaremos a eles na Seção 2.4.

A ideia central desta seção é a seguinte:

Em C, o tipo faz parte da definição da variável e determina como seus bits devem ser interpretados.

Ou seja: os mesmos 4 bytes na memória podem representar um número inteiro ou um número em ponto flutuante, dependendo exclusivamente do tipo declarado. O tipo não é um detalhe burocrático é o que dá significado aos bits armazenados.

2.3 Representação dos Dados na Memória

Este é um dos pontos em que C se diferencia claramente de linguagens de mais alto nível: podemos e devemos pensar em como os dados são fisicamente armazenados.

Considere a declaração:

```
1 int x = 10;
```

Na maioria dos sistemas atuais, um `int` ocupa **4 bytes** (32 bits) de memória. O valor 10, representado em binário, fica armazenado assim:

00000000	00000000	00000000	00001010
----------	----------	----------	----------

4 bytes

Não vamos entrar em detalhes sobre representação de números negativos (complemento de dois) neste momento isso terá seu espaço mais à frente. O que importa agora é fixar uma cadeia de raciocínio que vai acompanhar todo o restante da apostila:

variável → tipo → bytes → memória.

Toda variável que você declara reserva um espaço de memória cujo tamanho é determinado pelo seu tipo. Essa ideia é a base sobre a qual construiremos, mais adiante, os capítulos de ponteiros e de *structs* então vale a pena parar um instante e garantir que essa imagem mental ficou clara antes de avançar.

2.4 Constantes e Modificadores

Às vezes queremos declarar uma variável cujo valor não deve mudar ao longo do programa. Para isso, usamos a palavra-chave **const**:

```
1 const int idade = 20;
```

Qualquer tentativa de modificar `idade` depois disso resultará em erro de compilação o que é útil para proteger valores que representam configurações fixas, limites ou constantes matemáticas.

C também oferece **modificadores de tipo**, que ajustam o tamanho ou a forma como um número é interpretado:

```
1 unsigned int // apenas valores não negativos
2 long int     // inteiro com mais bits (maior faixa de valores)
3 short int    // inteiro com menos bits (faixa menor)
```

Não vamos nos aprofundar nesses modificadores agora pois eles terão uma seção própria mais adiante, quando discutirmos faixas de valores e *overflow* com mais detalhe.

2.5 Operadores

Operadores são símbolos que indicam ao compilador que operação deve ser realizada sobre um ou mais valores (chamados de *operandos*).

2.5.1 Operadores Aritméticos

```
1 + soma
2 - subtracao
3 * multiplicacao
4 / divisao
5 % modulo (resto da divisao inteira)
```

2.5.2 Operadores Relacionais

Usados para comparar dois valores, sempre resultando em verdadeiro (1) ou falso (0):

```
1 <      menor que
2 >      maior que
3 <=     menor ou igual
4 >=     maior ou igual
5 ==     igual a
6 !=     diferente de
```

2.5.3 Operadores Lógicos

Usados para combinar expressões relacionais:

```
1 &&     e (AND)
2 ||     ou (OR)
3 !      negacao (NOT)
```

2.5.4 Operadores de Atribuição

```
1 =      atribuicao simples
2 +=     soma e atribui
3 -=     subtrai e atribui
4 *=     multiplica e atribui
5 /=     divide e atribui
6 %=     calcula o modulo e atribui
```

Por exemplo, `total += 5;` é equivalente a `total = total + 5;`.

2.5.5 Incremento e Decremento

```
1 ++     incremento
2 --     decremento
```

```
1 i++;
2 ++i;
```

Vale destacar algo que costuma passar despercebido: **++ é um operador, e não simplesmente uma abreviação textual de `i = i + 1`**. Essa distinção parece sutil

agora, mas vai se tornar muito relevante quando estudarmos ponteiros e a diferença entre as formas pré-fixada (`++i`) e pós-fixada (`i++`) em expressões mais complexas, como `vetor[i++]`.

2.6 Conversão de Tipos

Em C, é possível e às vezes necessário converter um valor de um tipo para outro. Essa conversão é chamada de *casting*.

```
1 int a = 5;
2 int b = 2;
3
4 double resultado = (double) a / b;
```

Aqui está uma diferença importante em relação ao Python: em Python, `5 / 2` já retorna `2.5` automaticamente. Em C, `a / b` sendo ambos `int` realiza uma **divisão inteira**, descartando a parte decimal e resultando em `2`. Para obter o resultado com casas decimais, é preciso converter explicitamente um dos operandos para `double` (ou `float`) antes da divisão, como feito no exemplo acima.

Conversões também acontecem de forma implícita, quando atribuímos um valor de um tipo a uma variável de outro tipo:

```
1 int x = 10;
2 double y = x;           // conversao implicita: int -> double (sem
                           perda de dados)
```

```
1 double x = 10.5;
2 int y = (int) x;        // conversao explicita: double -> int (
                           perde a parte decimal, y = 10)
```

De forma geral, converter de um tipo "menor" para um "maior" (como `int` para `double`) é seguro. O caminho inverso pode causar perda de informação, e por isso é uma boa prática torná-lo explícito com um *cast*, como em `(int) x`.

2.7 Entrada e Saída de Dados

No Capítulo 1, você já usou a função `printf` para exibir uma mensagem fixa na tela. Agora que conhecemos variáveis e tipos, podemos usá-la e sua "parceira" `scanf` de forma sistemática, para exibir e ler valores dinâmicos.

```
1 printf("%d", idade);  
2 scanf("%d", &idade);
```

Cada tipo de dado tem um **especificador de formato** correspondente, usado tanto em `printf` quanto em `scanf`:

```
1 %d    int  
2 %f    float  
3 %lf   double  
4 %c    char  
5 %s    string (vetor de char)
```

Erro comum

```
1 scanf("%d", idade);    // errado  
2 scanf("%d", &idade);   // correto
```

O `scanf` precisa saber **onde**, na memória, deve escrever o valor lido e não apenas qual valor a variável tem no momento. Por isso, é necessário passar o **endereço de memória** da variável, usando o operador `&` antes do nome dela. Omitir o `&` é um dos erros mais comuns entre quem está começando em C, e o compilador nem sempre vai te avisar com clareza sobre isso.

Isso naturalmente levanta uma pergunta:

"Por que preciso colocar `&`?"

A resposta completa para essa pergunta pertence ao capítulo sobre ponteiros, mais adiante. Por ora, fica o registro consciente de que existe algo acontecendo por trás dessa sintaxe que ainda vamos explorar em profundidade. É importante não esconder isso do aluno, mas também não se aprofundar antes da hora.

2.8 Escopo e Tempo de Vida

Toda variável tem um **escopo** a região do código onde ela é visível e pode ser usada e um **tempo de vida** o período durante o qual ela existe na memória.

- **Escopo local:** variáveis declaradas dentro de uma função (ou de um bloco delimitado por chaves) só existem e só podem ser acessadas dentro daquele bloco.

- **Escopo global:** variáveis declaradas fora de qualquer função, no topo do arquivo, podem ser acessadas por qualquer função do programa.

```
1 int contador_global = 0; // escopo global
2
3 int main(void) {
4     // escopo local, existe apenas dentro de main
5     int idade_local = 20;
6     return 0;
7 }
```

De forma geral, variáveis locais existem apenas enquanto a função em que foram declaradas está em execução assim que a função termina, esse espaço de memória é liberado. Variáveis globais, por outro lado, existem durante toda a execução do programa.

Resumo do Capítulo

Neste capítulo, deixamos de tratar variáveis como simples "caixas de valores" e passamos a enxergá-las como regiões de memória com tipo, tamanho e representação bem definidos. Vimos os tipos fundamentais de C, como declarar e inicializar variáveis, os principais operadores da linguagem, como converter entre tipos e como realizar entrada e saída básica de dados. Também demos os primeiros passos sobre escopo um assunto que vai se tornar cada vez mais importante conforme nossos programas crescerem.

Ao final deste capítulo, você deve ser capaz de:

- Declarar e inicializar variáveis em C.
- Identificar os principais tipos fundamentais.
- Explicar a relação entre tipos e representação dos dados na memória.
- Utilizar operadores aritméticos, relacionais e lógicos.
- Realizar conversões de tipos.
- Utilizar `printf` e `scanf` para entrada e saída simples.
- Diferenciar escopo local e global.

Exercícios

Exercício

Fixação

Declare variáveis de tipos `int`, `float` e `char` para representar, respectivamente, a idade, a altura (em metros) e a inicial do primeiro nome de uma pessoa. Inicialize cada uma com um valor de sua escolha e utilize `printf` para exibir os três valores na tela, cada um com o especificador de formato adequado.

Aplicação

Escreva um programa que leia, via `scanf`, dois números inteiros digitados pelo usuário. O programa deve calcular e exibir a soma, a diferença, o produto e o resto da divisão entre eles.

Desafio

Analise o programa a seguir:

```
1 int a = 5;
2 int b = 2;
3
4 printf("%d\n", a / b);
5 printf("%f\n", (double)a / b);
```

Execute mentalmente (ou compile e rode, se preferir) esse trecho e responda: **por que as duas operações produzem resultados diferentes?**

Tente formular sua resposta usando os conceitos de tipo e representação de dados discutidos neste capítulo e não apenas como uma regra decorada sobre operadores.

Material Complementar

Neste capítulo foram utilizados diversos exemplos práticos. As implementações completas, listas adicionais de exercícios, slides da monitoria e perguntas frequentes encontram-se disponíveis no repositório da disciplina.

3 Controle de Fluxo

Até agora, todos os nossos programas seguiram um caminho único: a execução começa na primeira linha de `main` e segue, sequencialmente, até a última. Isso é suficiente para calcular expressões e exibir resultados, mas está longe de representar a maioria dos problemas reais. Precisamos que o computador seja capaz de **tomar decisões** ("se a senha estiver correta, libere o acesso") e de **repetir ações** ("some cada nota digitada até o usuário informar que terminou").

É exatamente isso que as estruturas de **controle de fluxo** nos permitem fazer: alterar a ordem natural de execução das instruções, com base em condições que só podem ser avaliadas durante a execução do programa. Neste capítulo, vamos estudar as duas grandes famílias dessas estruturas em C: as **condicionais**, que desviam o fluxo, e os **laços** (ou *loops*), que o repetem.

3.1 Condicionais

Estruturas condicionais permitem que um trecho de código seja executado apenas quando uma determinada condição for verdadeira. Em C, uma condição é sempre uma expressão que resulta em um valor: 0 é considerado falso, e qualquer valor diferente de zero é considerado verdadeiro.

3.1.1 A estrutura `if`

A forma mais simples de decisão é o `if`:

```
1 int idade = 18;
2
3 if (idade >= 18) {
4     printf("Maior de idade\n");
5 }
```

O bloco entre chaves só é executado se a condição dentro dos parênteses for verdadeira. Se a condição for falsa, o bloco é simplesmente ignorado e a execução continua na linha seguinte ao `if`.

3.1.2 if-else

Quando queremos executar um bloco alternativo caso a condição seja falsa, usamos `else`:

```
1 int idade = 15;
2
3 if (idade >= 18) {
4     printf("Maior de idade\n");
5 } else {
6     printf("Menor de idade\n");
7 }
```

3.1.3 Encadeando condições com else if

Quando existem múltiplas condições mutuamente exclusivas, podemos encadeá-las:

```
1 int nota = 7;
2
3 if (nota >= 9) {
4     printf("Conceito A\n");
5 } else if (nota >= 7) {
6     printf("Conceito B\n");
7 } else if (nota >= 5) {
8     printf("Conceito C\n");
9 } else {
10     printf("Reprovado\n");
11 }
```

É importante notar que as condições são avaliadas **em ordem**, de cima para baixo, e a execução para no primeiro bloco cuja condição for verdadeira. Trocar a ordem das comparações acima (por exemplo, testar `nota >= 5` antes de `nota >= 7`) mudaria completamente o comportamento do programa.

Erro comum

Confundir o operador de atribuição (=) com o operador de igualdade (==) dentro de uma condição:

```
1 int senha = 1234;
2
3 if (senha = 0) {    // errado: atribui 0 a senha, e a
4     condicao vira falsa
5     printf("Acesso negado\n");
6 }
7
8 if (senha == 0) {   // correto: compara senha com 0
9     printf("Acesso negado\n");
10 }
```

O trecho `senha = 0` não é uma comparação é uma atribuição, que **sobrescreve** o valor de `senha` e ainda avalia como falso (porque o valor atribuído foi 0). Esse é um dos erros mais traiçoeiros em C, porque o compilador geralmente não o considera um erro de sintaxe, apenas um possível aviso (*warning*). **Sempre revise suas condições com atenção redobrada.**

3.1.4 Aninhamento e chaves opcionais

Blocos de `if` podem conter outros `if` dentro deles, formando condicionais aninhadas:

```
1 if (idade >= 18) {
2     if (possui_documento) {
3         printf("Acesso liberado\n");
4     } else {
5         printf("Apresente um documento\n");
6     }
7 } else {
8     printf("Acesso negado: menor de idade\n");
9 }
```

Quando o bloco do `if` (ou do `else`) contém apenas uma única instrução, as chaves são tecnicamente opcionais:

```
1 if (idade >= 18)
2     printf("Maior de idade\n");
```

Embora válido, **recomendamos fortemente sempre usar chaves**, mesmo em blocos de uma linha só. Isso evita um erro clássico: se alguém adicionar uma segunda linha ao bloco no futuro, sem perceber que faltam chaves, apenas a primeira linha continuará condicionada ao `if` as demais serão executadas incondicionalmente, o que pode gerar bugs difíceis de detectar.

3.1.5 O operador ternário

Para condições simples que resultam na atribuição de um valor, C oferece uma forma compacta: o **operador ternário** `?`:

```
1 int idade = 20;
2 char *status = (idade >= 18) ? "maior" : "menor";
```

A expressão `condicao ? valor_se_verdadeiro : valor_se_falso` é equivalente a um `if-else` que atribui um valor a uma variável, mas escrita em uma única linha. É útil para casos simples, mas deve ser evitada quando a lógica se torna muito complexa nesse caso, um `if-else` tradicional costuma ser mais legível.

3.1.6 A estrutura switch

Quando uma mesma variável precisa ser comparada com vários valores possíveis, o `switch` pode ser uma alternativa mais organizada do que uma longa cadeia de `else if`:

```
1 int dia = 3;
2
3 switch (dia) {
4     case 1:
5         printf("Domingo\n");
6         break;
7     case 2:
8         printf("Segunda-feira\n");
9         break;
10    case 3:
11        printf("Terça-feira\n");
12        break;
13    default:
14        printf("Dia inválido\n");
15 }
```

O `switch` compara o valor da expressão (`dia`) com cada `case`, executando o bloco correspondente. O `default` é executado quando nenhum `case` corresponde ao valor de forma semelhante a um `else final`.

Erro comum

Esquecer o `break` ao final de cada `case`:

```
1 switch (dia) {  
2     case 1:  
3         printf("Domingo\n");  
4     case 2:  
5         printf("Segunda-feira\n"); // sera executado mesmo  
6         se dia == 1!  
7         break;  
8 }
```

Sem o `break`, a execução de um `switch` **continua** para o próximo `case`, executando-o também, independentemente da condição um comportamento chamado de *fall-through*. Às vezes esse comportamento é usado intencionalmente, mas na grande maioria dos casos ele é um erro de digitação com consequências sérias. Revise sempre se cada `case` termina com `break` (ou `return`, quando dentro de uma função).

O `switch` em C só funciona com valores de tipos inteiros (incluindo `char`) e não aceita, por exemplo, comparações com `strings` ou intervalos de valores. Limitações importantes a se ter em mente ao decidir entre um `switch` e uma cadeia de `if-else`.

3.2 Laços

Laços (também chamados de *loops*) permitem repetir um bloco de instruções enquanto uma condição for satisfeita, evitando que precisemos repetir manualmente o mesmo código várias vezes.

3.2.1 O laço while

O `while` repete um bloco de código **enquanto** sua condição for verdadeira. A condição é testada **antes** de cada repetição:

```
1 int contador = 1;  
2
```

```
3 while (contador <= 5) {  
4     printf("%d\n", contador);  
5     contador++;  
6 }
```

Se a condição for falsa já na primeira verificação, o bloco do **while** nunca é executado.

3.2.2 O laço do-while

O **do-while** é semelhante ao **while**, mas testa a condição **depois** de executar o bloco o que garante que o bloco seja executado pelo menos uma vez:

```
1 int opcao;  
2  
3 do {  
4     printf("Digite 0 para sair: ");  
5     scanf("%d", &opcao);  
6 } while (opcao != 0);
```

Essa estrutura é particularmente útil em situações como menus interativos, em que queremos executar a ação (exibir o menu) antes mesmo de sabermos se o usuário quer continuar ou não.

3.2.3 O laço for

O **for** é a estrutura de repetição mais utilizada quando sabemos, de antemão, quantas vezes (ou sob que condição controlada) o laço deve se repetir. Ele reúne três partes em uma única linha: inicialização, condição e incremento (ou decremento).

```
1 for (int i = 1; i <= 5; i++) {  
2     printf("%d\n", i);  
3 }
```

As três partes do **for** são executadas na seguinte ordem lógica:

1. A **inicialização** (`int i = 1`) é executada uma única vez, no início.
2. A **condição** (`i <= 5`) é testada antes de cada repetição. Se for falsa, o laço termina.
3. O bloco de código é executado.

4. O **incremento** (`i++`) é executado ao final de cada repetição, e o processo volta ao passo 2.

O `for` e o `while` são, no fundo, formas equivalentes de expressar a mesma ideia o exemplo acima poderia ser reescrito com um `while`:

```
1 int i = 1;
2 while (i <= 5) {
3     printf("%d\n", i);
4     i++;
5 }
```

Na prática, usamos `for` quando o número de repetições (ou o critério de parada) é conhecido de antemão, e `while` quando a repetição depende de uma condição que só se revela durante a execução do programa (como esperar por uma entrada específica do usuário).

Erro comum

Criar um laço infinito por esquecer de atualizar a variável de controle:

```
1 int i = 1;
2 while (i <= 5) {
3     printf("%d\n", i);
4     // faltou i++; o laço nunca termina!
5 }
```

Todo laço controlado por condição precisa de algo que, eventualmente, torne essa condição falsa. Antes de compilar um laço `while` ou `do-while`, pergunte-se sempre: **"o que, dentro deste bloco, está caminhando para tornar a condição falsa?"** Se a resposta for "nada", você encontrou um laço infinito antes mesmo de executá-lo.

3.2.4 `break` e `continue`

Duas palavras-chave permitem alterar o comportamento padrão de um laço:

- **`break`** interrompe o laço imediatamente, saindo dele por completo.
- **`continue`** interrompe apenas a repetição atual, pulando direto para a próxima verificação da condição.

```
1 for (int i = 1; i <= 10; i++) {
2     if (i == 5) {
3         break; // encerra o laço quando i for 5
4     }
5     printf("%d\n", i);
6 }
```

```
1 for (int i = 1; i <= 10; i++) {
2     if (i % 2 == 0) {
3         continue; // pula os numeros pares
4     }
5     printf("%d\n", i);
6 }
```

3.2.5 Laços aninhados

Assim como as condicionais, laços também podem ser aninhados — um laço dentro de outro. Isso é especialmente útil para percorrer estruturas bidimensionais, como veremos mais adiante ao estudarmos matrizes:

```
1 for (int i = 1; i <= 3; i++) {
2     for (int j = 1; j <= 3; j++) {
3         printf("(%d, %d) ", i, j);
4     }
5     printf("\n");
6 }
```

Note que, a cada repetição do laço externo (controlado por *i*), o laço interno (controlado por *j*) é executado **por completo** antes que o laço externo avance para a próxima repetição. Entender essa ordem de execução é fundamental para trabalhar corretamente com laços aninhados.

Resumo do Capítulo

Neste capítulo, aprendemos a alterar o fluxo sequencial padrão de um programa em C. Vimos como tomar decisões com `if`, `else if`, `else` e `switch`, e como repetir blocos de código com `while`, `do-while` e `for`. Também vimos como controlar o comportamento de um laço com `break` e `continue`, e discutimos armadilhas comuns, como confundir =

com `==` e esquecer de atualizar a condição de um laço. Essas estruturas são a espinha dorsal de praticamente todo programa não trivial, e vamos utilizá-las constantemente dos próximos capítulos em diante.

Ao final deste capítulo, você deve ser capaz de:

- Utilizar `if`, `else if` e `else` para tomar decisões.
- Utilizar o operador ternário em condições simples.
- Utilizar `switch` para comparar uma variável com múltiplos valores possíveis.
- Diferenciar `while`, `do-while` e `for`, e escolher a estrutura mais adequada a cada situação.
- Utilizar `break` e `continue` para controlar a execução de um laço.
- Identificar e evitar laços infinitos.
- Trabalhar com condicionais e laços aninhados.

Exercícios

Exercício 1 — Fixação

Escreva um programa que leia um número inteiro digitado pelo usuário e informe, usando `if-else`, se ele é positivo, negativo ou igual a zero.

Exercício 2 — Aplicação

Escreva um programa que utilize um laço `for` para exibir todos os números de 1 a 100 que sejam múltiplos de 3 **ou** de 5. Em seguida, utilizando um segundo laço, calcule e exiba a soma de todos esses números.

Exercício 3 — Desafio

Escreva um programa que simule um menu simples utilizando `do-while` e `switch`. O menu deve oferecer três opções (por exemplo, "1. Somar dois números", "2. Verificar se um número é par ou ímpar", "0. Sair") e continuar sendo exibido até que o usuário escolha a opção de sair.

Depois de implementar o programa, reflita: por que o **do-while** é mais adequado do que um **while** comum nesse cenário? O que mudaria no comportamento do programa se você tivesse usado um **while** no lugar?

4 Funções

Até aqui, todo o código dos nossos programas viveu dentro de uma única função: `main`. Isso funciona para programas pequenos, mas rapidamente se torna insustentável à medida que a complexidade cresce. Imagine um programa que precisa calcular a média de notas de uma turma em três momentos diferentes: copiar e colar o mesmo trecho de cálculo três vezes não só é repetitivo, como também é um convite a erros: se um dia você precisar corrigir a fórmula, terá que lembrar de alterá-la em todos os lugares onde ela foi copiada.

Funções resolvem exatamente esse problema. Uma função é um bloco de código nomeado, que pode ser **reutilizado** quantas vezes for necessário, recebendo diferentes entradas e produzindo diferentes saídas a cada chamada. Você já usa funções desde o Capítulo 1, mesmo sem ter percebido: `printf` e `scanf` nada mais são do que funções prontas, fornecidas pela biblioteca padrão de C. Neste capítulo, vamos aprender a criar as nossas próprias.

Organizar um programa em funções traz benefícios que vão muito além de "economizar digitação":

- **Reutilização:** um mesmo trecho de lógica pode ser chamado de vários pontos do programa.
- **Organização:** um problema grande pode ser dividido em subproblemas menores, cada um resolvido por uma função, retomando diretamente a ideia de decomposição de problemas que apresentamos no Capítulo 1.
- **Manutenção:** corrigir um erro ou melhorar um algoritmo em um único lugar automaticamente corrige ou melhora todos os pontos do programa que dependem dele.
- **Legibilidade:** um `main` composto por chamadas a funções bem nomeadas (como `calcular_media(notas)`) costuma ser muito mais fácil de entender do que um bloco único e extenso de instruções.

4.1 Declaração

Declarar (ou, mais precisamente, **definir**) uma função em C significa escrever seu código completo: seu nome, os dados que ela recebe, o tipo de dado que ela devolve e as instruções que ela executa.

A estrutura geral de uma função é:

```
1 tipo_de_retorno nome_da_funcao(parametros) {  
2     // corpo da funcao  
3     return valor; // se houver retorno  
4 }
```

Vamos ver um exemplo completo, de uma função que soma dois números inteiros:

```
1 int somar(int a, int b) {  
2     int resultado = a + b;  
3     return resultado;  
4 }
```

Dissecando essa declaração:

- `int` (antes do nome) é o **tipo de retorno** o tipo do valor que a função devolve para quem a chamou.
- `somar` é o **nome** da função, seguindo as mesmas regras de identificadores já vistas para variáveis, no Capítulo 2.
- `(int a, int b)` é a **lista de parâmetros** os dados que a função recebe como entrada, cada um com seu próprio tipo e nome.
- `return resultado;` devolve o valor calculado para quem chamou a função, e encerra sua execução naquele ponto.

Para **chamar** (ou **invocar**) essa função a partir de `main`, usamos seu nome seguido dos valores desejados entre parênteses:

```
1 int main(void) {  
2     int total = somar(5, 3);  
3     printf("%d\n", total); // exibe 8  
4     return 0;  
5 }
```

Os valores 5 e 3, passados na chamada, são copiados para os parâmetros `a` e `b` dentro da função voltaremos a esse mecanismo com mais detalhe na Seção 4.3.

4.1.1 Funções sem retorno: void

Nem toda função precisa devolver um valor. Quando uma função apenas realiza uma ação (como exibir algo na tela), sem produzir um resultado a ser usado depois, seu tipo de retorno é `void`:

```
1 void saudar(char nome[]) {  
2     printf("Ola, %s!\n", nome);  
3 }
```

Uma função `void` pode usar `return`; (sem nenhum valor após a palavra-chave) para encerrar sua execução antecipadamente, mas não pode usar `return` seguido de um valor.

4.1.2 Funções sem parâmetros

Quando uma função não precisa receber nenhuma entrada, indicamos isso com `void` dentro dos parênteses exatamente como já fazíamos com `main`, desde o Capítulo 1:

```
1 void exibir_menu(void) {  
2     printf("1. Iniciar\n");  
3     printf("2. Sair\n");  
4 }
```

Erro comum

Esquecer o `return` em uma função que promete devolver um valor, ou tentar usar o resultado de uma função `void` como se ela retornasse algo:

```
1 int dobro(int x) {  
2     int resultado = x * 2;  
3     // faltou o return!  
4 }  
5  
6 int main(void) {  
7     int y = dobro(5); // comportamento indefinido: dobro  
8                     // nao devolveu nada  
9     return 0;  
}
```

Se uma função é declarada com um tipo de retorno diferente de `void`, ela **precisa** garantir um `return` com um valor compatível em todos os caminhos possíveis

de execução. Muitos compiladores emitem apenas um *warning* nesse caso, e não um erro o que torna esse descuido especialmente perigoso, já que o programa compila normalmente, mas se comporta de forma imprevisível.

4.2 Protótipos

Em C, o compilador lê o arquivo de cima para baixo, uma única vez. Isso significa que, se uma função for chamada em um ponto do código **antes** de sua definição completa aparecer, o compilador não terá como saber, naquele momento, quantos parâmetros ela espera ou qual valor ela devolve.

```
1 int main(void) {
2     int total = somar(5, 3); // erro: somar ainda nao foi
        declarada!
3     return 0;
4 }
5
6 int somar(int a, int b) {
7     return a + b;
8 }
```

A solução para esse problema é o **protótipo** de função: uma declaração antecipada, contendo apenas a "assinatura" da função (tipo de retorno, nome e tipos dos parâmetros), sem o corpo. O protótipo é colocado no início do arquivo, antes de `main`, avisando o compilador sobre a existência da função antes mesmo de sua definição completa ter sido lida:

```
1 int somar(int a, int b); // prototipo
2
3 int main(void) {
4     int total = somar(5, 3); // agora o compilador ja conhece
        somar
5     printf("%d\n", total);
6     return 0;
7 }
8
9 int somar(int a, int b) { // definicao completa
10     return a + b;
11 }
```

Note que o protótipo termina com ponto e vírgula, e não possui corpo — ele é apenas uma promessa de que a função será definida em algum lugar do programa. Os nomes dos parâmetros no protótipo são opcionais (o compilador só se importa com os tipos), então também seria válido escrever apenas `int somar(int, int);`; mas é uma boa prática manter os nomes, já que eles funcionam como documentação de uso da função.

Erro comum

Declarar um protótipo com uma assinatura e depois definir a função com outra, diferente:

```
1 int somar(int a, int b); // prototipo promete devolver int
2
3 void somar(int a, int b) { // definicao devolve void:
4     incompatibilidade!
5     printf("%d\n", a + b);
6 }
```

O protótipo e a definição da função precisam ser absolutamente consistentes quanto ao tipo de retorno e aos tipos dos parâmetros. Divergências como essa resultam em erro de compilação, e a mensagem gerada pelo compilador costuma apontar diretamente para essa incompatibilidade — vale a pena aprender a interpretá-la.

Na prática, à medida que os programas crescem, é comum organizar os protótipos das funções em **arquivos de cabeçalho** (*header files*, com extensão `.h`), permitindo que várias partes de um projeto compartilhem as mesmas declarações. Essa organização mais avançada será retomada quando estudarmos como dividir um programa em múltiplos arquivos-fonte.

4.3 Passagem por Valor

Quando você chama uma função e passa argumentos para ela, como em `somar(5, 3)`, o que exatamente acontece com esses valores dentro da função? Em C, o mecanismo padrão de passagem de argumentos é chamado de **passagem por valor**.

Isso significa que, ao chamar uma função, C **copia** o valor de cada argumento para dentro do parâmetro correspondente. A função passa a trabalhar com essa cópia local, e **qualquer alteração feita no parâmetro dentro da função não afeta a variável original**, de fora da função.

Vamos observar isso com cuidado:

```
1 void dobrar(int numero) {
2     numero = numero * 2;
3     printf("Dentro da funcao: %d\n", numero);
4 }
5
6 int main(void) {
7     int valor = 10;
8     dobrar(valor);
9     printf("Fora da funcao: %d\n", valor);
10    return 0;
11 }
```

A saída desse programa é:

```
1 Dentro da funcao: 20
2 Fora da funcao: 10
```

Mesmo depois de chamar `dobrar(valor)`, a variável `valor` em `main` continua com o valor 10. Isso acontece porque `numero`, dentro da função `dobrar`, é uma **variável completamente separada** de `valor` — ela apenas recebeu, no momento da chamada, uma **cópia** do valor armazenado em `valor`. Alterar `numero` altera apenas essa cópia local, que deixa de existir assim que a função termina.

main	dobrar
valor = 10	numero = 10 (cópia) numero = 20 (apos o dobro)
valor continua 10	numero deixa de existir

Esse comportamento é consistente com tudo o que discutimos no Capítulo 2 sobre variáveis e memória: cada variável ocupa seu próprio espaço, e uma atribuição como `numero = numero * 2` apenas modifica o conteúdo do espaço de memória de `numero`, sem qualquer relação com o espaço de memória de `valor`.

Erro comum

Esperar que uma função modifique, "por fora", uma variável passada como argumento:

```
1 void zerar(int x) {
2     x = 0;
3 }
4
```

```
5 int main(void) {  
6     int saldo = 100;  
7     zerar(saldo);  
8     printf("%d\n", saldo); // ainda imprime 100, e nao 0!  
9     return 0;  
10 }
```

Essa é uma das maiores fontes de confusão para quem está começando em C. A função `zerar` realmente zera sua cópia local de `x`, mas isso não tem nenhum efeito sobre `saldo`, em `main`. Se o objetivo é que uma função altere o valor de uma variável que existe fora dela, a passagem por valor **não é suficiente** será necessário um mecanismo diferente.

E é exatamente aqui que fechamos a Parte I da apostila com uma pergunta em aberto, no mesmo espírito da que deixamos no Capítulo 2 sobre o `&` do `scanf`:

Se a passagem por valor sempre trabalha com cópias, como faz o `scanf` para efetivamente alterar o valor de uma variável declarada em `main`? E como poderíamos escrever nossas próprias funções capazes de fazer o mesmo?

A resposta para essas perguntas é a peça que finalmente vai unir tudo o que vimos sobre memória, o operador `&` e a passagem de argumentos é o assunto central da Parte II da apostila, que começa com o estudo de **ponteiros**.

Resumo do Capítulo

Neste capítulo, aprendemos a organizar nosso código em funções: blocos reutilizáveis, capazes de receber entradas (parâmetros) e produzir saídas (valores de retorno). Vimos como declarar e definir uma função, por que protótipos são necessários para que o compilador conheça uma função antes de sua definição completa, e como funciona a passagem por valor o mecanismo padrão de C para enviar argumentos a uma função, sempre por meio de cópias. Encerramos com uma pergunta propositalmente sem resposta completa, que vai nos guiar diretamente ao próximo grande tema da apostila: ponteiros.

Ao final deste capítulo, você deve ser capaz de:

- Declarar e definir funções com e sem retorno, com e sem parâmetros.
- Explicar por que e quando um protótipo de função é necessário.
- Diferenciar protótipo e definição de uma função.
- Explicar o funcionamento da passagem por valor em C.
- Prever corretamente se uma alteração feita dentro de uma função afeta ou não a variável original, fora dela.

Exercícios

Exercício 1 — Fixação

Escreva uma função `int quadrado(int n)` que devolva o quadrado do número recebido como parâmetro. Escreva também o protótipo correspondente, e chame a função a partir de `main` para exibir o quadrado de um número digitado pelo usuário.

Exercício 2 — Aplicação

Escreva um programa com três funções: `ler_notas`, responsável por ler três notas digitadas pelo usuário; `calcular_media`, que recebe as três notas e devolve a média entre elas; e `exibir_resultado`, que recebe a média e exibe se o aluno foi aprovado (média maior ou igual a 6) ou reprovado. Utilize protótipos para todas as funções antes de `main`.

Exercício 3 — Desafio

Considere o programa a seguir:

```
1 void trocar(int a, int b) {  
2     int temp = a;  
3     a = b;  
4     b = temp;  
5 }  
6  
7 int main(void) {
```

```
8     int x = 5, y = 10;
9     trocar(x, y);
10    printf("x=%d, y=%d\n", x, y);
11    return 0;
12 }
```

Qual será a saída exibida por esse programa? Explique, com base no que aprendemos sobre passagem por valor, por que a função **trocar não consegue** trocar de fato os valores de **x** e **y** em **main**. O que você imagina que precisaria mudar, na declaração da função e em sua chamada, para que essa troca realmente funcionasse?

Parte II

Organização dos Dados

5 Vetores

Até agora, cada variável que declaramos guardava exatamente um valor: uma idade, uma nota, um resultado de soma. Mas a maioria dos problemas reais envolve **coleções** de dados relacionados: as notas de uma turma inteira, os nomes de uma lista de convidados, as temperaturas registradas ao longo de uma semana. Guardar cada um desses valores em uma variável separada (`nota1`, `nota2`, `nota3`, ...) rapidamente se torna inviável e, pior, impede que você processe esses dados com um laço, já que não existe como "percorrer" um conjunto de variáveis independentes.

Neste capítulo, damos o primeiro passo da Parte II da apostila, dedicada à **organização de dados**: vamos aprender a usar **vetores**, a estrutura mais fundamental de C para armazenar múltiplos valores do mesmo tipo sob um único nome.

5.1 Conceito de Vetor

Um **vetor** é uma sequência de elementos do mesmo tipo, armazenados de forma **contígua** na memória ou seja, um logo em seguida do outro, sem espaços entre eles. Cada elemento é acessado por meio de um **índice**, que indica sua posição dentro do vetor.

<code>notas[0]</code>	<code>notas[1]</code>	<code>notas[2]</code>	<code>notas[3]</code>	<code>notas[4]</code>
7	8	5	9	6

Um detalhe fundamental, que costuma confundir quem vem de fora da programação: em C, os índices de um vetor **começam em 0**, e não em 1 (*zero-based numbering*). Existem algumas razões interessantes para a adoção dessa regra, vale a pena conferir. Por isso, um vetor com 5 elementos possui índices válidos de 0 a 4 — não existe um `notas[5]`.

Como os elementos ficam lado a lado na memória, conhecer o endereço do primeiro elemento e o tamanho de cada elemento (determinado pelo seu tipo) é suficiente para calcular o endereço de qualquer outro elemento do vetor. Essa propriedade é o que

torna o acesso a um vetor extremamente rápido, e é também a base da relação entre vetores e ponteiros, que exploraremos na Seção 5.6.

5.2 Declaração e Inicialização

Para declarar um vetor, informamos o tipo dos elementos, um nome e, entre colchetes, quantos elementos ele deve conter:

```
1 int notas[5];
```

Isso reserva espaço para 5 números inteiros consecutivos, todos ainda não inicializados (contendo "lixo de memória", exatamente como discutimos para variáveis simples no Capítulo 2).

Também é possível inicializar um vetor no momento da declaração, informando seus valores entre chaves:

```
1 int valores[] = {10, 20, 30, 40};
```

Nesse caso, o tamanho do vetor é **implícito**: o compilador conta os valores fornecidos e reserva exatamente o espaço necessário aqui, 4 posições.

Também é possível combinar tamanho explícito com uma lista de inicialização menor que o tamanho total, o que chamamos de **inicialização parcial**:

```
1 int valores[5] = {10, 20}; // valores[2], valores[3] e valores
    [4] viram 0
```

Quando isso acontece, todas as posições não explicitamente inicializadas são automaticamente preenchidas com 0 um dos poucos casos em C em que uma variável recebe um valor inicial "de graça".

5.3 Acesso aos Elementos

Acessamos (para leitura ou alteração) um elemento específico de um vetor usando o operador de índice []:

```
1 int notas[5] = {7, 8, 5, 9, 6};
2
3 printf("%d\n", notas[0]); // le o primeiro elemento: 7
4 notas[2] = 10;           // altera o terceiro elemento
```

C **não verifica automaticamente** se um índice está dentro dos limites válidos do vetor. Acessar uma posição inexistente não gera um erro de compilação nem, necessariamente, um erro perceptível durante a execução — o programa pode simplesmente ler ou escrever em uma região de memória que não pertence ao vetor, com consequências imprevisíveis: desde um valor sem sentido até a corrupção de outras variáveis ou o encerramento abrupto do programa.

Erro comum

Acessar a posição igual ao tamanho do vetor, por acreditar (erroneamente) que os índices vão de 1 até o tamanho:

```
1 int notas[5] = {7, 8, 5, 9, 6};
2
3 printf("%d\n", notas[5]); // fora dos limites! índices
    validos: 0 a 4
```

Um vetor de tamanho n sempre tem índices válidos de 0 a $n - 1$. Essa é provavelmente a fonte de erro mais comum entre quem está começando a trabalhar com vetores em C e, diferente de linguagens como Python, o programa não interrompe a execução com uma mensagem clara de erro: o comportamento é apenas *indefinido*.

5.4 Percorrendo Vetores

Como os elementos de um vetor têm índices sequenciais, um laço `for` é a ferramenta natural para acessar todos eles, um de cada vez:

```
1 int notas[5] = {7, 8, 5, 9, 6};
2
3 for (int i = 0; i < 5; i++) {
4     printf("%d\n", notas[i]);
5 }
```

Note a condição do laço: $i < 5$, e não $i \leq 5$ — garantindo que o último índice acessado seja 4, o último válido.

Esse padrão — percorrer um vetor com um laço — é a base para praticamente todas as operações que faremos daqui em diante, como somar todos os elementos usando um **acumulador**:

```
1 int soma = 0;
```

```
2 for (int i = 0; i < 5; i++) {
3     soma += notas[i];
4 }
```

ou contar quantos elementos satisfazem uma condição, usando um **contador**:

```
1 int aprovados = 0;
2 for (int i = 0; i < 5; i++) {
3     if (notas[i] >= 6) {
4         aprovados++;
5     }
6 }
```

5.5 Vetores e Funções

Assim como variáveis, vetores podem ser passados como argumentos para funções. A sintaxe do parâmetro, no entanto, é um pouco diferente:

```
1 void exibir(int vetor[], int tamanho) {
2     for (int i = 0; i < tamanho; i++) {
3         printf("%d\n", vetor[i]);
4     }
5 }
```

Um detalhe crucial: **o tamanho do vetor não "viaja" junto com ele** quando passado para uma função. Diferente do que acontece dentro do próprio arquivo, onde o compilador sabe que `notas` tem 5 elementos porque viu sua declaração, uma função que recebe um vetor como parâmetro só enxerga o endereço do seu primeiro elemento ela não tem como saber, sozinha, onde ele termina. Por isso, é uma convenção quase universal em C passar o tamanho do vetor como um **segundo parâmetro**, como fizemos acima com `tamanho`.

Outro ponto importante: diferente da passagem por valor que estudamos no Capítulo 4, **alterações feitas nos elementos de um vetor dentro de uma função afetam o vetor original**, fora dela:

```
1 void zerar_vetor(int vetor[], int tamanho) {
2     for (int i = 0; i < tamanho; i++) {
3         vetor[i] = 0;
4     }
5 }
```

```
6
7 int main(void) {
8     int valores[3] = {1, 2, 3};
9     zerar_vetor(valores, 3);
10    printf("%d\n", valores[0]); // exibe 0!
11    return 0;
12 }
```

Isso pode parecer contraditório com o que aprendemos sobre passagem por valor e é exatamente essa aparente contradição que vamos explicar na próxima seção.

5.6 Vetores e Ponteiros

Chegamos a uma das conexões mais importantes da linguagem C. O nome de um vetor, usado sozinho em uma expressão, é tratado pelo compilador como o **endereço do seu primeiro elemento**. Ou seja, as três expressões abaixo são equivalentes:

```
1 notas           // endereço do primeiro elemento
2 &notas[0]        // endereço do primeiro elemento, explicitamente
```

É exatamente por isso que, ao passar um vetor para uma função, você está na prática passando apenas um **endereço de memória**, e não uma cópia de todos os elementos. É esse mecanismo que explica o comportamento observado na Seção 5.5: a função recebe o endereço de onde o vetor *de verdade* está armazenado, e por isso qualquer alteração feita através desse endereço afeta o vetor original.

C também permite acessar os elementos de um vetor usando **aritmética de ponteiros**: somar um número ao endereço do vetor avança essa quantidade de *elementos* (não de bytes) dentro dele. Isso significa que a notação familiar `notas[i]` é, internamente, apenas uma forma mais legível de escrever `*(notas + i)`:

```
1 notas[i]        // forma usual
2 *(notas + i)     // forma equivalente, usando aritmetica de
                   ponteiros
```

Não é necessário, neste momento, dominar completamente a manipulação de ponteiros — esse será o tema central da Parte III da apostila. O objetivo aqui é apenas construir a intuição correta: **um vetor é, fundamentalmente, um endereço de memória associado a um tipo e a um tamanho conhecido em tempo de compilação**. Guarde essa ideia — ela vai reaparecer, com muito mais profundidade, quando estudarmos ponteiros e alocação dinâmica.

5.7 Operações Clássicas sobre Vetores

Grande parte dos problemas envolvendo vetores se resume a variações de um pequeno conjunto de operações. Vale a pena conhecê-las bem, pois elas reaparecem constantemente:

- **Soma e média:** percorrer o vetor acumulando os valores e, ao final, dividir pela quantidade de elementos.
- **Maior e menor elemento:** percorrer o vetor guardando, em uma variável auxiliar, o maior (ou menor) valor encontrado até o momento.
- **Busca:** percorrer o vetor procurando um valor específico, geralmente interrompendo o laço com `break` assim que ele é encontrado.
- **Contagem de ocorrências:** percorrer o vetor contando quantas vezes um valor (ou uma condição) aparece.
- **Inversão:** trocar os elementos das extremidades para o centro, geralmente usando dois índices que se aproximam.
- **Cópia:** percorrer um vetor de origem, copiando cada elemento para as posições correspondentes de um vetor de destino.

Todas essas operações compartilham a mesma estrutura básica: um laço `for` que vai de 0 até `tamanho - 1`, com alguma lógica de acumulação, comparação ou cópia dentro dele. Dominar esse padrão é mais valioso do que decorar cada operação isoladamente.

Erro comum

Usar um vetor sem garantir que todas as posições relevantes foram inicializadas antes de lê-las:

```
1 int valores[5]; // nenhum elemento inicializado
2
3 int soma = 0;
4 for (int i = 0; i < 5; i++) {
5     soma += valores[i]; // soma "lixo de memoria"
6 }
```

Assim como uma variável simples, um vetor declarado localmente (dentro de uma função) **não** começa automaticamente zerado. Se o seu algoritmo depende de valores conhecidos antes de processá-los, garanta explicitamente que o vetor

foi preenchido (seja por inicialização direta, seja por leitura via `scanf`) antes de usá-lo em cálculos.

5.8 Erros Comuns

Reunimos aqui, de forma resumida, as armadilhas mais frequentes ao trabalhar com vetores em C:

- Assumir que os índices começam em 1, em vez de 0.
- Acessar `vetor[tamanho]`, uma posição sempre inválida (o último índice válido é `tamanho - 1`).
- Confundir o **tamanho** do vetor com o **último índice** válido.
- Esquecer de passar o tamanho do vetor como parâmetro ao usá-lo em uma função.
- Processar um vetor cujas posições ainda não foram inicializadas.

Resumo do Capítulo

Neste capítulo, demos o primeiro passo da Parte II ao aprender a trabalhar com coleções de dados por meio de vetores. Vimos como declarar, inicializar, acessar e percorrer vetores, como passá-los para funções e por que, diferente de variáveis comuns, alterações feitas dentro da função afetam o vetor original. Também introduzimos, de forma intuitiva, a relação entre vetores e endereços de memória, uma ideia que será retomada com profundidade na Parte III. No próximo capítulo, vamos aplicar tudo isso a um caso particular extremamente importante: vetores de caracteres, usados para representar texto.

Ao final deste capítulo, você deve ser capaz de:

- Explicar o que é um vetor e por que seus índices começam em 0.
- Declarar e inicializar vetores, total ou parcialmente.
- Acessar e alterar elementos de um vetor com segurança.
- Percorrer um vetor com `for`, implementando acumuladores e contadores.
- Passar vetores para funções, incluindo seu tamanho como parâmetro.
- Explicar por que alterações em um vetor dentro de uma função afetam o vetor original.
- Implementar as operações clássicas sobre vetores: soma, média, busca, maior/menor elemento, contagem, inversão e cópia.

Exercícios

Exercício 1 — Fixação

Declare um vetor de 10 inteiros, preencha-o lendo valores digitados pelo usuário e exiba, em seguida, todos os elementos em ordem inversa (do último para o primeiro).

Exercício 2 — Aplicação

Escreva um programa que leia a quantidade de alunos de uma turma, limitada a 50, e armazene suas notas em um vetor. Ao final, exiba a média da turma, a maior nota e a menor nota.

Exercício 3 — Aplicação

Escreva uma função `int buscar(int vetor[], int tamanho, int valor)` que devolva o índice da primeira ocorrência de `valor` dentro do vetor, ou `-1` caso ele não seja encontrado. Utilize essa função em um pequeno programa de teste.

Exercício 4 — Integrador

Escreva um programa que leia um vetor de números inteiros e, utilizando funções separadas para cada tarefa, seja capaz de:

1. remover as duplicatas, criando um novo vetor contendo apenas valores únicos;
2. ordenar o novo vetor em ordem crescente, utilizando o algoritmo de ordenação por seleção ou por bolha;
3. exibir o resultado final.

Exercício 5 — Desafio

Sem utilizar uma função de biblioteca pronta, implemente uma função

```
void inverter(int vetor[], int tamanho)
```

que inverta a ordem dos elementos de um vetor **no próprio vetor**, ou seja, sem criar um vetor auxiliar.

Dica: utilize dois índices: um começando no início do vetor e outro no final. A cada repetição, troque os elementos apontados pelos índices e faça-os se aproximarem até que se encontrem.

6 Strings

No capítulo anterior, aprendemos a trabalhar com vetores: sequências de elementos do mesmo tipo, armazenados de forma contígua na memória. Neste capítulo, vamos aplicar exatamente esses mesmos conceitos a um caso particular extremamente importante a representação de texto.

É fundamental deixar uma ideia clara desde o início: **C não possui um tipo primitivo chamado `string`**. Diferente de Python, Java ou JavaScript, onde texto é um tipo de dado nativo com seus próprios operadores e métodos, em C uma string é simplesmente um **vetor de `char`**, seguindo uma convenção específica que vamos entender neste capítulo. Tudo o que aprendemos sobre vetores — declaração, índices, percurso com `for`, passagem para funções — se aplica diretamente aqui.

6.1 O que é uma String em C?

Uma string, em C, é um vetor de caracteres (`char`) que termina com um caractere especial, chamado **terminador nulo**, representado por `'\0'`. Esse terminador marca onde o texto 'real' acaba dentro do vetor.

```
1 char nome[20];
```

Esse trecho reserva espaço para até 20 caracteres, mas isso não significa que a string armazenada ali usará necessariamente todo esse espaço. O que importa não é o tamanho do vetor, e sim onde o `'\0'` aparece dentro dele.

6.2 Inicialização de Strings

A forma mais comum de inicializar uma string é por meio de uma **string literal**, entre aspas duplas:

```
1 char nome[] = "Rafael";
```

Essa linha é, na prática, equivalente a inicializar manualmente um vetor de `char`, caractere por caractere, incluindo o terminador nulo ao final:

```
1 char nome[] = {'R', 'a', 'f', 'a', 'e', 'l', '\0'};
```

Note que o vetor `nome` tem, na verdade, **7 posições**, e não 6 a string literal "Rafael" inclui automaticamente o `'\0'` ao final, ainda que ele não apareça explicitamente entre as aspas. Esquecer desse caractere extra é uma fonte comum de erros ao reservar espaço manualmente para strings, como veremos na Seção 6.8.

6.3 O Caractere `'\0'`

O terminador nulo merece atenção especial, porque é a peça que sustenta todo o funcionamento de strings em C.

Como uma string é apenas um vetor de `char`, e um vetor, por si só, não "sabe" quantos de seus elementos estão efetivamente em uso, funções como `printf` e `strlen` precisam de alguma forma de saber onde o texto termina. A convenção adotada por C é: **o texto vai do primeiro caractere até, mas sem incluir, o primeiro `'\0'` encontrado**. Tudo que estiver no vetor depois disso é ignorado por essas funções, mesmo que o vetor tenha espaço reservado além desse ponto.

Erro comum

Confundir o caractere `'\0'` (terminador nulo, valor numérico 0) com o caractere `'0'` (o dígito zero, valor numérico 48 na tabela ASCII):

```
1 char digito = '0';    // o caractere que representa o dígito
    zero
2 char fim = '\0';      // o terminador nulo, usado para
    marcar o fim de uma string
```

Visualmente parecidos, esses dois caracteres têm significados completamente diferentes. Usar `'0'` onde deveria estar `'\0'` é um erro sutil: o compilador não vai reclamar, mas funções que dependem do terminador nulo para saber onde parar (como `printf` com `%s`) vão continuar lendo memória além do que você pretendia.

Uma consequência direta disso: o **tamanho do vetor** que armazena uma string quase sempre precisa ser **maior** do que o número de caracteres "visíveis" do texto, para acomodar também o `'\0'`. Um vetor de 20 posições, como o declarado na Seção 6.1, pode armazenar textos de até 19 caracteres a vigésima posição precisa sobrar para o terminador.

6.4 Entrada e Saída de Strings

Já usamos `printf` para exibir strings desde o Capítulo 1, com o especificador `%s`:

```
1 char nome[] = "Rafael";  
2 printf("Ola, %s!\n", nome);
```

Para ler uma string digitada pelo usuário, também podemos usar `scanf` com `%s`:

```
1 char nome[20];  
2 scanf("%s", nome);
```

Note que, diferente do que fizemos com `int` e `float` no Capítulo 2, **não usamos o `&` antes de `nome`**. Isso acontece porque, como vimos na Seção 5.6, o nome de um vetor já representa o endereço do seu primeiro elemento e é exatamente esse endereço que `scanf` precisa receber.

O `scanf` com `%s`, no entanto, tem uma limitação importante: ele interrompe a leitura no primeiro espaço em branco, o que o torna inadequado para ler frases inteiras (como "Rafael Silva"). Além disso, ele não tem nenhuma noção do tamanho do vetor de destino, o que abre espaço para o vetor receber mais caracteres do que ele comporta.

Uma alternativa mais adequada para ler linhas completas de texto é a função `fgets`:

```
1 char frase[50];  
2 fgets(frase, 50, stdin);
```

Diferente de `scanf`, `fgets` recebe explicitamente o tamanho máximo do vetor de destino, o que o torna uma escolha mais segura. Não vamos nos aprofundar agora em todas as nuances de entrada segura de texto o objetivo desta seção é apenas entender, de forma geral, como strings entram e saem de um programa em C.

6.5 Funções da Biblioteca `<string.h>`

C não tem operadores nativos para trabalhar com strings não é possível, por exemplo, comparar duas strings com `==` ou concatená-las com `+`, como em outras linguagens. Para isso, usamos as funções fornecidas pela biblioteca `<string.h>`. As mais importantes, neste momento, são:

- `strlen(s)`: devolve o número de caracteres de `s`, **sem contar** o terminador nulo.
- `strcpy(destino, origem)`: copia o conteúdo de `origem` para `destino`, incluindo o terminador nulo.

- `strncpy(destino, origem, n)`: semelhante a `strcpy`, mas copia no máximo `n` caracteres uma alternativa mais segura quando o tamanho de `destino` é limitado.
- `strcmp(s1, s2)`: compara `s1` e `s2`, devolvendo 0 se forem iguais (e um valor diferente de zero caso contrário).
- `strcat(destino, origem)`: concatena `origem` ao final de `destino`.

```
1 #include <string.h>
2
3 char nome[20] = "Rafael";
4 printf("%d\n", strlen(nome)); // exibe 6
5
6 char copia[20];
7 strcpy(copia, nome);
8
9 if (strcmp(nome, copia) == 0) {
10     printf("As strings são iguais\n");
11 }
```

O objetivo aqui não é decorar essa lista, e sim entender **o que cada função faz de fato** com o vetor de caracteres por trás da string. Em particular, é importante notar que `strcpy` e `strcat` **escrevem** no vetor de destino, e por isso ele precisa ter espaço suficiente reservado previamente, ou o programa corre o risco de escrever além dos limites do vetor.

6.6 Strings e Funções

Assim como vetores em geral, strings são passadas para funções por meio do endereço de seu primeiro caractere um parâmetro do tipo `char *` (que estudaremos formalmente na Parte III, mas que já podemos usar de forma intuitiva agora):

```
1 void exibir_maiusculo(char texto[]) {
2     for (int i = 0; texto[i] != '\0'; i++) {
3         printf("%c", toupper(texto[i]));
4     }
5     printf("\n");
6 }
```

Quando uma função apenas **lê** uma string, sem modificá-la, é uma boa prática declarar o parâmetro como `const char *`, sinalizando tanto para o compilador quanto para quem lê o código que a função não deve (e não vai) alterar o conteúdo recebido:

```
1 void exibir(const char texto[]) {  
2     printf("%s\n", texto);  
3 }
```

Se, por outro lado, a função precisa **devolver** uma nova string, é preciso ter cuidado especial: uma string criada dentro de uma função, em um vetor local, deixa de existir assim que a função termina. Devolver o endereço desse vetor resultaria em um endereço inválido. Esse problema só será completamente resolvido quando estudarmos alocação dinâmica de memória, na Parte III; por ora, o caminho mais simples é fazer a função escrever o resultado diretamente em um vetor fornecido pelo chamador, como fizemos com `strcpy`.

6.7 Percorrendo Strings Manualmente

Como uma string é apenas um vetor terminado em `'\0'`, podemos percorrê-la sem sequer conhecer seu tamanho de antemão: basta avançar enquanto o caractere atual não for o terminador:

```
1 for (int i = 0; texto[i] != '\0'; i++) {  
2     // processa texto[i]  
3 }
```

Esse padrão é extremamente comum e serve de base para praticamente qualquer processamento de texto "na mão", sem depender de `<string.h>`:

- Contar quantos caracteres uma string possui (reimplementando `strlen`).
- Contar quantas vogais aparecem em um texto.
- Converter cada caractere para maiúsculo ou minúsculo.
- Procurar a primeira ocorrência de um caractere específico.

Por exemplo, uma implementação manual de `strlen`:

```
1 int meu_strlen(const char texto[]) {  
2     int contador = 0;  
3     while (texto[contador] != '\0') {
```

```
4         contador++;  
5     }  
6     return contador;  
7 }
```

Implementar essas funções "do zero" mesmo sabendo que versões prontas existem em `<string.h>` é um exercício particularmente valioso para consolidar a ideia de que uma string não é uma "caixa mágica de texto", mas sim um vetor comum, percorrido com a mesma lógica que já dominamos no Capítulo 5.

6.8 Erros Comuns

- Esquecer de reservar espaço para o `'\0'` ao dimensionar um vetor de `char` manualmente.
- Confundir `'A'` (um único `char`) com `"A"` (uma string de um caractere, que na verdade ocupa duas posições: `'A'` e `'\0'`).
- Reservar espaço insuficiente para o resultado de `strcpy` ou `strcat`, escrevendo além dos limites do vetor de destino.
- Usar `sizeof` esperando obter o comprimento da string: `sizeof` devolve o tamanho **do vetor** (em bytes), não o número de caracteres úteis do texto que é exatamente o que `strlen` calcula.
- Ultrapassar o tamanho do vetor ao ler uma entrada maior do que o esperado com `scanf`.
- Confundir o nome de um vetor de `char` com um ponteiro genérico, esquecendo que ele sempre aponta para um espaço de memória de tamanho fixo, definido na declaração.

Resumo do Capítulo

Neste capítulo, entendemos que strings em C não são um tipo à parte, mas sim uma aplicação direta do que aprendemos sobre vetores no capítulo anterior: um vetor de `char`, terminado pelo caractere especial `'\0'`. Vimos como inicializar, ler e exibir strings, conhecemos as principais funções de `<string.h>` e, mais importante, aprendemos a percorrer uma string manualmente uma habilidade que reforça a compreensão

de que texto, em C, é simplesmente memória organizada de uma forma específica. No próximo capítulo, damos um passo além na organização de dados, estendendo a ideia de vetor para mais de uma dimensão.

Ao final deste capítulo, você deve ser capaz de:

- Explicar por que uma string em C é, na verdade, um vetor de `char`.
- Explicar o papel do caractere `'\0'` na representação de strings.
- Inicializar, ler e exibir strings corretamente.
- Utilizar as principais funções de `<string.h>`: `strlen`, `strcpy`, `strncpy`, `strcmp` e `strcat`.
- Passar strings para funções, diferenciando parâmetros que apenas leem (`const char *`) de parâmetros que escrevem.
- Percorrer uma string manualmente usando seu terminador nulo como condição de parada.
- Reconhecer e evitar os erros mais comuns ao manipular strings.

Exercícios

Exercício 1 — Fixação

Declare uma string, leia um nome digitado pelo usuário com `scanf` e exiba, em seguida, quantos caracteres ele possui, usando `strlen`.

Exercício 2 — Aplicação

Escreva uma função `int contar_vogais(const char texto[])` que percorra a string manualmente (sem usar `<string.h>`) e devolva quantas vogais ela contém, considerando maiúsculas e minúsculas.

Exercício 3 — Implementação Clássica

Implemente, sem usar `<string.h>`, uma função `int minha_strcmp(const char s1[], const char s2[])` que se comporte como a função `strcmp` da biblioteca padrão: devolvendo 0 se as strings forem iguais, e um valor diferente de zero caso contrário.

Exercício 4 — Integrador

Escreva um programa que leia uma frase (usando `fgets`) e, usando funções separadas para cada tarefa, seja capaz de: (a) contar o número de palavras da frase (considerando espaços como separadores); (b) converter a frase inteira para maiúsculas; (c) verificar se a frase é um palíndromo, ignorando espaços.

Exercício 5 — Desafio

Sem usar `strcpy`, implemente uma função `void minha_strcpy(char destino[], const char origem[])` que copie o conteúdo de `origem` para `destino`, caractere por caractere, garantindo que o terminador nulo também seja copiado corretamente.

7 Arrays Multidimensionais

No Capítulo 5, aprendemos a organizar uma sequência de valores em um vetor. No Capítulo 6, vimos que uma string nada mais é do que um caso particular de vetor — um vetor de `char`. Neste capítulo, damos mais um passo natural nessa mesma linha de raciocínio, em vez de introduzir um conceito inteiramente novo:

Se um vetor é uma sequência de elementos, o que acontece quando cada elemento dessa sequência é, ele próprio, um vetor?

A resposta é uma **matriz** (ou, de forma mais geral, um **array multidimensional**): uma estrutura organizada em mais de uma dimensão, útil sempre que os dados que você precisa representar têm, naturalmente, mais de um "eixo" como linhas e colunas de uma tabela, ou as casas de um tabuleiro.

7.1 Conceito de Arrays Multidimensionais

A forma mais comum de array multidimensional é a **matriz bidimensional**: um vetor de vetores, organizado em **linhas** e **colunas**.

```
1 int matriz[3][4];
```

Essa declaração cria uma matriz com 3 linhas e 4 colunas — ou seja, 12 elementos ao todo, todos do tipo `int`. Assim como em vetores comuns, os índices de linhas e colunas também começam em 0: a matriz acima possui linhas válidas de 0 a 2, e colunas válidas de 0 a 3.

	col 0	col 1	col 2	col 3
lin 0	[0][0]	[0][1]	[0][2]	[0][3]
lin 1	[1][0]	[1][1]	[1][2]	[1][3]
lin 2	[2][0]	[2][1]	[2][2]	[2][3]

Embora tenhamos apresentado o caso bidimensional, o mesmo princípio se estende para três ou mais dimensões (`int cubo[2][3][4]`; por exemplo) mas, na grande maioria dos problemas práticos, duas dimensões já são suficientes, e é nelas que vamos focar neste capítulo.

7.2 Declaração e Inicialização

Uma matriz pode ser inicializada no momento da sua declaração, usando listas aninhadas — uma lista externa, representando as linhas, e uma lista interna para cada uma delas, representando as colunas:

```
1 int matriz[2][3] = {  
2     {1, 2, 3},  
3     {4, 5, 6}  
4 };
```

Assim como em vetores unidimensionais, também é possível fazer uma **inicialização parcial**, deixando o compilador preencher com 0 as posições não especificadas:

```
1 int matriz[2][3] = {  
2     {1, 2},          // matriz[0][2] vira 0  
3     {4}              // matriz[1][1] e matriz[1][2] viram 0  
4 };
```

Uma particularidade da declaração de matrizes: quando inicializamos a matriz na própria declaração, o tamanho da **primeira** dimensão (o número de linhas) pode ser omitido, pois o compilador consegue deduzi-lo a partir da quantidade de listas internas fornecidas — mas o tamanho da segunda dimensão em diante **sempre** precisa ser informado explicitamente:

```
1 int matriz[][3] = {    // numero de linhas deduzido: 2  
2     {1, 2, 3},  
3     {4, 5, 6}  
4 };
```

Matrizes com o mesmo número de linhas e colunas são chamadas de **matrizes quadradas**, e aparecem com frequência em problemas envolvendo tabuleiros ou transformações matemáticas, como veremos na Seção 7.7.

7.3 Acesso aos Elementos

Para acessar um elemento específico de uma matriz, usamos dois índices entre colchetes: o primeiro indicando a linha, o segundo indicando a coluna.

```
1 int matriz[2][3] = {  
2     {1, 2, 3},  
3     {4, 5, 6}  
4 };  
5  
6 printf("%d\n", matriz[1][2]); // acessa a linha 1, coluna 2:  
    // exibe 6  
7 matriz[0][0] = 100;           // altera a linha 0, coluna 0
```

A ordem dos índices importa: `matriz[1][2]` e `matriz[2][1]` referem-se a posições diferentes (e a segunda sequer seria válida na matriz do exemplo acima, que só tem 3 colunas). Um erro comum é inverter mentalmente essa ordem, especialmente ao traduzir um problema descrito em termos de "linha X, coluna Y" para código: vamos retomar esse cuidado na Seção 7.9.

7.4 Percorrendo Matrizes

Para visitar todos os elementos de uma matriz, usamos **laços aninhados**: um laço externo percorrendo as linhas, e um laço interno percorrendo as colunas de cada linha, retomando diretamente o que estudamos no Capítulo 3:

```
1 for (int i = 0; i < linhas; i++) {  
2     for (int j = 0; j < colunas; j++) {  
3         printf("%d_", matriz[i][j]);  
4     }  
5     printf("\n");  
6 }
```

É importante internalizar a ordem de execução desse padrão: para cada valor de `i` (cada linha), o laço interno percorre **todas** as colunas antes que `i` avance para a linha seguinte. Esse é exatamente o mesmo comportamento que já discutimos ao apresentar laços aninhados no Capítulo 3: a diferença agora é que temos uma estrutura de dados real, a matriz, para dar sentido a essa dupla repetição.

7.5 Matrizes e Memória

Apesar de pensarmos em uma matriz como uma grade bidimensional, a memória do computador é, fisicamente, **unidimensional** um longo corredor de bytes, um após o outro. Isso significa que, por trás da notação `matriz[i][j]`, o compilador precisa converter dois índices em um único deslocamento dentro desse corredor contínuo de memória.

Em C, essa conversão segue a chamada **ordem row-major** (ordem por linhas): todos os elementos da linha 0 são armazenados primeiro, seguidos por todos os elementos da linha 1, e assim por diante.

[0][0]	[0][1]	[0][2]	[1][0]	[1][1]	[1][2]
--------	--------	--------	--------	--------	--------

matriz 2D armazenada como um bloco contínuo de 6 elementos

Essa organização explica por que, ao declarar um parâmetro de matriz em uma função (Seção 7.6), o número de **colunas** precisa ser sempre conhecido: é essa informação que permite ao compilador calcular corretamente o deslocamento até qualquer elemento `matriz[i][j]`, mesmo enxergando a memória apenas como uma sequência contínua de valores.

Não vamos nos aprofundar, por ora, em como alocar matrizes **dinamicamente** (isto é, com dimensões definidas apenas durante a execução do programa) esse tema pertence à Parte III, quando já tivermos discutido ponteiros e alocação de memória com a profundidade necessária.

7.6 Matrizes e Funções

Assim como vetores unidimensionais, matrizes são passadas para funções por meio do endereço do seu primeiro elemento e, também aqui, o compilador não carrega consigo as dimensões da matriz automaticamente. A diferença em relação a vetores simples é que, para matrizes, **a segunda dimensão em diante precisa ser informada diretamente na assinatura da função**:

```

1 void imprime_matriz(int matriz[][3], int linhas) {
2     for (int i = 0; i < linhas; i++) {
3         for (int j = 0; j < 3; j++) {
4             printf("%d ", matriz[i][j]);
5         }
6         printf("\n");
    }
}
```

```
7     }  
8 }
```

Isso acontece exatamente pelo motivo discutido na Seção 7.5: para calcular o endereço de `matriz[i][j]` dentro do bloco contínuo de memória, o compilador precisa saber quantas colunas cada linha possui sem essa informação, seria impossível determinar onde uma linha termina e a próxima começa. O número de **linhas**, por outro lado, pode ser passado normalmente como um parâmetro separado, assim como fazíamos com o tamanho de um vetor unidimensional no Capítulo 5.

7.7 Operações Clássicas

Assim como fizemos para vetores, vale a pena conhecer bem um pequeno conjunto de operações que aparecem repetidamente em problemas envolvendo matrizes:

- **Soma de matrizes:** percorrer ambas as matrizes com os mesmos índices `i` e `j`, somando os elementos correspondentes em uma matriz resultado.
- **Multiplicação por escalar:** percorrer a matriz multiplicando cada elemento por um valor fixo.
- **Transposição:** criar uma nova matriz em que a linha `i` da matriz original se torna a coluna `i` da nova matriz (ou seja, `transposta[j][i] = original[i][j]`).
- **Diagonal principal:** os elementos em que o índice da linha é igual ao da coluna (`matriz[i][i]`), presentes apenas em matrizes quadradas.
- **Maior elemento:** percorrer todos os elementos com laços aninhados, guardando o maior valor encontrado até o momento a mesma lógica usada em vetores, agora com dois índices.
- **Soma de linhas/colunas:** acumular, separadamente, a soma de cada linha (ou de cada coluna) da matriz.

Como você deve ter notado, todas essas operações reaproveitam diretamente o padrão de laços aninhados apresentado na Seção 7.4 a principal diferença entre elas está em **quais índices variam e o que é feito com `matriz[i][j]`** a cada repetição.

7.8 Matrizes como Aplicações

Embora seja comum apresentar matrizes por meio de operações matemáticas, sua utilidade vai muito além disso. Vale a pena ter em mente que uma matriz é, fundamentalmente, apenas **uma forma de organizar dados em duas dimensões** o que a torna adequada para representar diversas situações do mundo real:

- Um **tabuleiro** de jogo (como jogo da velha ou batalha naval), em que cada posição `matriz[i][j]` representa uma casa.
- Uma **tabela de notas**, em que cada linha representa um aluno e cada coluna uma avaliação diferente.
- Uma **imagem simplificada em escala de cinza**, em que cada posição da matriz armazena a intensidade de um pixel.
- Um **mapa ou grade**, representando, por exemplo, obstáculos e caminhos livres em um pequeno labirinto.

Reconhecer esses padrões é útil não apenas para resolver exercícios, mas para identificar, em problemas futuros, quando uma matriz é a estrutura de dados mais natural para representar a informação em questão.

7.9 Erros Comuns

- Confundir linha e coluna ao acessar um elemento, escrevendo `matriz[coluna][linha]` em vez de `matriz[linha][coluna]`.
- Inverter as dimensões ao declarar a matriz (por exemplo, declarar `int matriz[4][3]` quando o problema exige 3 linhas e 4 colunas).
- Ultrapassar os limites válidos de linha ou de coluna, de forma semelhante ao que já vimos com vetores no Capítulo 5.
- Assumir que os índices de linha e coluna começam em 1, em vez de 0.
- Errar a condição de um dos dois laços `for` (por exemplo, usar o número de linhas como limite do laço que deveria percorrer as colunas).
- Passar o número de colunas incorretamente (ou esquecê-lo) ao declarar uma função que recebe uma matriz como parâmetro.

Resumo do Capítulo

Neste capítulo, encerramos a Parte II estendendo a ideia de vetor para mais de uma dimensão. Vimos que uma matriz pode ser entendida, de forma direta, como um vetor de vetores, organizado em linhas e colunas. Aprendemos a declarar, inicializar, acessar e percorrer matrizes com laços aninhados, entendemos como elas são efetivamente armazenadas na memória (de forma contígua, em ordem por linhas) e como passá-las corretamente para funções. Terminamos vendo que uma matriz é, acima de tudo, uma ferramenta de organização de dados, aplicável a problemas que vão muito além do cálculo puramente matemático. A partir daqui, com vetores, strings e matrizes bem consolidados, estamos prontos para a Parte III da apostila, que vai revisitar tudo o que vimos sob uma nova perspectiva: *onde exatamente esses dados estão na memória, e como podemos manipulá-los diretamente através de seus endereços?*

Ao final deste capítulo, você deve ser capaz de:

- Explicar o que é uma matriz e sua relação com vetores de vetores.
- Declarar e inicializar matrizes, total ou parcialmente.
- Acessar e alterar elementos de uma matriz usando os índices corretos de linha e coluna.
- Percorrer uma matriz utilizando laços aninhados.
- Explicar como uma matriz bidimensional é armazenada em uma memória fisicamente unidimensional.
- Passar matrizes para funções, informando corretamente o número de colunas.
- Implementar operações clássicas sobre matrizes: soma, transposição, diagonal principal, maior elemento e soma de linhas/colunas.
- Reconhecer situações do mundo real que podem ser representadas por uma matriz.

Exercícios

Exercício 1 — Fixação

Declare uma matriz 3x3, preencha-a lendo valores digitados pelo usuário e exiba-a formatada em linhas e colunas, como uma tabela.

Exercício 2 — Aplicação

Escreva um programa que leia uma matriz 4x4 de inteiros e calcule, usando funções separadas: (a) a soma de todos os elementos; (b) a soma dos elementos da diagonal principal; (c) o maior elemento da matriz e sua posição (linha e coluna).

Exercício 3 — Aplicação

Escreva uma função `void transpor(int origem[][3], int destino[][2], int linhas, int colunas)` que receba uma matriz de `linhas x colunas` e produza sua transposta (`colunas x linhas`) em `destino`. Teste com uma matriz 2x3 de sua escolha.

Exercício 4 — Integrador

Implemente o jogo da velha para dois jogadores no terminal: represente o tabuleiro como uma matriz 3x3 de `char`, exiba-o formatado a cada jogada, leia a posição desejada por cada jogador (linha e coluna) e verifique, a cada rodada, se algum jogador venceu ou se houve empate.

Exercício 5 — Desafio

Escreva um programa que represente uma imagem simplificada em escala de cinza como uma matriz de inteiros (valores de 0 a 255). Implemente uma função que "espelhe" a imagem horizontalmente (invertendo a ordem dos elementos em cada linha) e outra que calcule o valor médio de intensidade de toda a imagem.

Parte III

Gerenciamento de Memória

8 Ponteiros

```
1 int x = 10;  
2 int *p = &x;
```

Importante

Nunca utilize um ponteiro sem inicializá-lo.

9 Organização da Memória

9.1 Segmento de Código

9.2 Stack

9.3 Heap

9.4 Data

9.5 BSS

10 Alocação Dinâmica

Exercícios

Parte IV

Programação Modular

11 Structs e Enums

12 Modularização

12.1 Arquivos Header

12.2 Include Guards

12.3 Compilação Separada

12.4 Linkagem

13 Estruturas de Dados Básicas

Parte V

Tópicos Avançados

14 Manipulação Bit a Bit

15 Recursão

16 Depuração

Parte VI

Para ir além

Para ir além

Este conteúdo não é necessário para acompanhar o fluxo principal da disciplina, mas explora conceitos importantes para compreender melhor como os computadores representam e manipulam dados.

Tudo o que foi apresentado até aqui é suficiente para escrever programas sólidos e corretos em C. Essa parte especial existe para quem termina a apostila com aquela sensação de: *"Eu quero entender o que está acontecendo por baixo dos panos"*, ou *"Por que as coisas são desse jeito e não de outro?"*

Os capítulos a seguir não seguem uma ordem estritamente sequencial como as Partes I, II e III: cada um pode ser lido de forma relativamente independente, de acordo com sua curiosidade. Alguns aprofundam temas que apenas tocamos de leve ao longo do curso (como a representação de texto); outros exploram cantos da linguagem que, embora não sejam indispensáveis no dia a dia, revelam bastante sobre como C – e, por extensão, o próprio computador – realmente funciona.

Boa parte do conteúdo desta seção dialoga diretamente com o livro *Understanding and Using C Pointers*, de Richard Reese (O'Reilly), uma referência natural para quem já concluiu a Parte III e quer se aprofundar especificamente em ponteiros e gerenciamento de memória. Recomendamos sua leitura complementar a quem se interessar pelos capítulos finais desta parte.

Representação de Texto: do ASCII ao Unicode

No Capítulo 6, tratamos strings da forma como o dia a dia da programação em C exige: um vetor de `char` terminado em `'\0'`, percorrido caractere a caractere. Essa definição é suficiente para praticamente tudo que você vai programar durante o curso. Mas ela também esconde, deliberadamente, uma pergunta incômoda: **o que exatamente é um "caractere"?**

Este capítulo assume esse convite. Vamos percorrer o caminho histórico e técnico que vai do ASCII - um esquema simples, pensado para o inglês e para teleimpressoras dos anos 1960 - até o Unicode, o padrão que hoje tenta representar, em um único sistema, praticamente toda a escrita humana. No caminho, vamos descobrir que a intuição de "um caractere é um byte", que funciona bem o suficiente para não atrapalhar os primeiros capítulos, é uma simplificação que deixa de se sustentar assim que você sai do inglês.

Por que representar texto?

Um computador, no fundo, só manipula números. Toda vez que você vê a letra `'A'` na tela, o que existe de fato, na memória, é um número e é uma **convenção**, não uma lei física, que decide que esse número específico deve ser desenhado na tela como a letra "A".

Essa convenção precisa ser compartilhada. Se o seu programa grava o número 65 esperando que ele signifique "A", e outro programa lê esse mesmo número esperando que ele signifique outra coisa, o resultado é um arquivo de texto ilegível, cheio de símbolos sem sentido – um problema historicamente conhecido como *mojibake*. A necessidade de um padrão comum de representação de texto não é um detalhe acadêmico: é o que torna possível que um e-mail escrito no Brasil seja lido corretamente no Japão, ou que um código-fonte compartilhado no GitHub compile igualmente em qualquer parte do

mundo.

É essa necessidade que motiva toda a história que vamos contar neste capítulo.

ASCII e a representação de caracteres

O **ASCII** (*American Standard Code for Information Interchange*), padronizado em 1963, foi uma das primeiras tentativas amplamente adotadas de resolver esse problema. Sua proposta era simples: usar **7 bits** para representar cada caractere, o que permite codificar até $2^7 = 128$ símbolos diferentes — suficiente para as letras maiúsculas e minúsculas do alfabeto inglês, os dez dígitos, sinais de pontuação e alguns caracteres de controle (como o próprio `'\0'`, o `'\n'` de nova linha, e o `'\t'` de tabulação).

Caractere	Decimal	Binário
'A'	65	01000001
'a'	97	01100001
'0'	48	00110000
'\n'	10	00001010
'\0'	0	00000000

É exatamente essa tabela que sustenta a linha `char letra = 'A';` do Capítulo 2: internamente, `letra` guarda o número 65, e é apenas por convenção — reforçada pelo especificador `%c` de `printf` — que esse número é exibido como a letra "A" na tela.

Como C reserva, no mínimo, 8 bits (1 byte) para um `char`, sobra sempre 1 bit não utilizado pelo ASCII original. Diferentes fabricantes de computadores aproveitaram esse bit extra para criar suas próprias extensões — como o **Latin-1** (ISO 8859-1), que usa os 256 valores possíveis de um byte para acomodar caracteres acentuados comuns em línguas europeias, como o português. O problema é que essas extensões não eram compatíveis entre si: o mesmo número podia representar um "ç" em uma extensão e um símbolo completamente diferente em outra — e é exatamente esse cenário fragmentado que preparou o terreno para o Unicode.

Unicode e *code points*

O **Unicode** nasceu com um objetivo ambicioso: criar um único padrão capaz de representar, em teoria, todo sistema de escrita já usado pela humanidade — do alfabeto latino aos ideogramas do chinês, passando por hebraico, árabe, cirílico e até emojis.

Para isso, o Unicode abandona a ideia de "um byte = um caractere". Em vez disso, ele associa a cada caractere um número único, chamado **code point**, geralmente escrito no formato U+XXXX (em hexadecimal). Por exemplo:

Caractere	Code point
'A'	U+0041
'ç'	U+00E7
'¥'	U+00A5
'🤪'	U+1F600

Note que os primeiros 128 *code points* do Unicode (U+0000 a U+007F) coincidem, propositalmente, com a tabela ASCII original – uma decisão de compatibilidade retroativa que facilita muito a transição de sistemas antigos. Mas o Unicode vai muito além: sua versão mais recente já define mais de 149 mil *code points*, cobrindo dezenas de sistemas de escrita.

É crucial entender que o Unicode, por si só, **não define como esses números devem ser armazenados em bytes na memória ou em um arquivo**. Um *code point* como U+1F600 claramente não cabe em um único byte (que só vai até 255). Essa é a lacuna que as **codificações** (*encodings*) – como UTF-8, que veremos a seguir – vêm preencher.

UTF-8 e codificação de tamanho variável

O **UTF-8** é, hoje, a codificação Unicode mais usada na internet e em sistemas operacionais modernos. Sua ideia central é representar cada *code point* usando **de 1 a 4 bytes**, dependendo do seu valor:

Faixa do code point	Bytes usados	Exemplo
U+0000 a U+007F	1 byte	'A'
U+0080 a U+07FF	2 bytes	'ç'
U+0800 a U+FFFF	3 bytes	' '
U+10000 a U+10FFFF	4 bytes	' [GRINNING FACE]' (U+1F600)

A grande vantagem dessa abordagem é a **compatibilidade retroativa com ASCII**: qualquer texto composto exclusivamente por caracteres ASCII (os primeiros 128) é, byte a byte, idêntico em UTF-8. É por isso que um programa em C escrito e testado apenas com texto em inglês costuma "funcionar por acaso" mesmo sem qualquer preocupação especial com codificação até o momento em que um "ç", um "á" ou um emoji entra em cena.

Isso também explica por que o UTF-8 é chamado de codificação de **tamanho variável**: diferentes caracteres ocupam diferentes quantidades de bytes. Essa característica tem uma consequência direta e importante para quem programa em C, que exploramos na próxima seção.

Strings em C: bytes versus caracteres

Lembre-se da definição que fixamos no Capítulo 6: uma string em C é um vetor de `char`, e cada `char` ocupa exatamente 1 byte. Quando um arquivo-fonte em C contém uma string literal com acentuação, como `"Programação"`, salva em UTF-8 (o padrão da grande maioria dos editores modernos), essa string não é composta por 11 *caracteres visuais* e 11 `chars` correspondentes: ela é composta por 11 caracteres visuais, mas por **mais** de 11 bytes, porque o `"ç"` e o `"ã"` ocupam 2 bytes cada um em UTF-8.

```
1 char texto[] = "Programacao"; // sem acentos: 11 bytes + '\0'
2 char texto2[] = "Programação"; // com acentos: 13 bytes + '\0'
   em UTF-8
```

Isso quebra uma suposição implícita que carregamos, sem perceber, desde o Capítulo 6: a de que `strlen` e um laço `for` que percorre `texto[i]` sempre visitam "um caractere por vez". Na presença de UTF-8, `strlen("Programação")` não devolve 11: devolve o número de **bytes**, que é maior. E um laço ingênuo, percorrendo byte a byte, não vai visitar "um caractere de cada vez": vai, ocasionalmente, parar no **meio** de um caractere multibyte, tratando um pedaço dele como se fosse um caractere completo.

Erro comum

Assumir que `strlen` conta caracteres, quando na verdade ela conta bytes:

```
1 char cidade[] = "São Paulo";
2 printf("%d\n", strlen(cidade)); // NAO devolve 9!
```

Como o `"ã"` ocupa 2 bytes em UTF-8, `strlen` devolve um valor maior do que o número de caracteres visíveis na tela. Isso não é um bug de `strlen`: ela está fazendo exatamente o que promete (contar bytes até o `'\0'`). O erro está em esperar dela algo que ela nunca prometeu: contar caracteres em um sentido "humano".

Essa distinção bytes versus caracteres é a raiz de praticamente toda a complexidade que envolve texto internacionalizado em C, e é também o motivo pelo qual a linguagem oferece uma alternativa às strings de `char` comuns, que veremos a seguir.

wchar_t e *wide strings*

Para lidar com o problema de "um caractere por posição de vetor", C oferece o tipo `wchar_t` (definido em `<wchar.h>`), pensado para representar caracteres "largos" capazes, em teoria, de acomodar um *code point* inteiro em uma única posição, independentemente do quão grande ele seja.

```
1 #include <wchar.h>
2
3 wchar_t letra = L'a';
4 wchar_t frase[] = L"Programacao";
```

O prefixo `L` antes de um literal de caractere ou string (`L'a'`, `L"..."`) indica ao compilador que ele deve ser tratado como *wide*. Funções específicas como `wprintf` no lugar de `printf`, ou `wcslen` no lugar de `strlen` operam sobre esses tipos largos.

Aqui mora uma armadilha importante, e um dos motivos pelos quais este tema fica reservado para "Para ir além": o **tamanho de `wchar_t` não é definido pelo padrão C, e varia entre sistemas**. No Linux, `wchar_t` costuma ter 4 bytes, suficiente para qualquer *code point* Unicode em uma única posição (efetivamente, UTF-32). No Windows, por razões históricas, `wchar_t` tem apenas 2 bytes insuficiente para representar diretamente todo o Unicode, o que obriga o Windows a usar, internamente, uma variação de UTF-16 (que voltaremos a discutir na próxima seção). Um código que assume "1 `wchar_t` = 1 caractere, sempre" pode, portanto, funcionar perfeitamente em um sistema e falhar silenciosamente em outro.

UTF-8 x UTF-16 x UTF-32

Já vimos o UTF-8 em detalhe. Vale a pena colocar as três codificações Unicode mais comuns lado a lado, para entender os *trade-offs* de cada uma:

Codificação	Tamanho por caractere	Onde é comum
UTF-8	1 a 4 bytes (variável)	Web, Linux, arquivos-texto, C moderno
UTF-16	2 ou 4 bytes (variável)	Windows (<code>wchar_t</code>), Java, JavaScript
UTF-32	4 bytes (fixo)	Processamento interno, <code>wchar_t</code> no Linux

O UTF-32 é o mais simples de raciocinar: como todo *code point* ocupa exatamente 4 bytes, "acessar o caractere `i`" é uma operação de custo constante basta pular `i * 4` bytes. O preço dessa simplicidade é o desperdício de espaço: um texto inteiramente

em ASCII, que ocuparia 1 byte por caractere em UTF-8, ocupa 4 vezes mais espaço em UTF-32.

O UTF-16, adotado por sistemas mais antigos como o Windows e por linguagens como Java, tenta um meio-termo: a maioria dos caracteres do "plano básico" do Unicode cabe em 2 bytes, mas caracteres além disso (como muitos emojis) precisam de um mecanismo especial, chamado *surrogate pair*, que combina dois valores de 2 bytes para representar um único *code point* reintroduzindo, por outra via, o mesmo problema de tamanho variável que o UTF-32 tentava evitar.

O UTF-8 se tornou o padrão dominante na web e em sistemas Unix/Linux justamente por equilibrar bem esses fatores: é compacto para texto majoritariamente em inglês (o caso mais comum globalmente em protocolos e formatos de arquivo), compatível byte a byte com ASCII, e não depende de decidir a *ordem dos bytes* (*endianness*) um problema real que afeta tanto UTF-16 quanto UTF-32 ao trocar dados entre sistemas diferentes.

O que realmente significa "tamanho de uma string"?

Depois de todo esse percurso, vale reunir, em um só lugar, os diferentes sentidos possíveis da pergunta "qual é o tamanho desta string?" porque, como vimos, a resposta depende inteiramente do que você está de fato medindo:

- **Bytes:** quantos `char` o vetor ocupa, sem contar `'\0'`. É o que `strlen` devolve, e é o valor relevante para decidir se um buffer é grande o suficiente para receber a string.
- **Code points:** quantos caracteres Unicode distintos existem no texto. É o valor que, intuitivamente, esperamos como "quantidade de letras" mas que exige percorrer a string *decodificando* o UTF-8 corretamente, e não apenas contando bytes.
- **Clusters de grafemas:** o que um usuário humano perceberia como "um único caractere na tela". Alguns símbolos visuais como um emoji seguido de um modificador de tom de pele, ou uma letra seguida de um acento combinante são compostos por **múltiplos** *code points*, mas aparecem como uma única unidade visual. Esse é o nível mais complexo, e normalmente delegado a bibliotecas especializadas (como a ICU), e não implementado à mão.

Um mesmo texto pode, portanto, ter três respostas diferentes e igualmente "corretas" para "quantos caracteres ele tem", dependendo de qual dessas três definições você

está usando. A maior parte dos bugs de internacionalização em software nasce, precisamente, de confundir esses três níveis geralmente assumindo, sem perceber, que "bytes" e "caracteres visuais" são a mesma coisa, porque em textos puramente em inglês, com ASCII, eles de fato coincidem.

Considerações de portabilidade em C

Ao trabalhar com texto internacionalizado em C, alguns cuidados práticos ajudam a evitar boa parte das armadilhas discutidas neste capítulo:

- **Não assuma que 1 `char` = 1 caractere visual.** Se seu programa precisa processar corretamente acentos, emojis ou alfabetos não latinos "caractere por caractere", um laço simples sobre um vetor de `char` não é suficiente — é necessário decodificar o UTF-8 corretamente, ou recorrer a uma biblioteca especializada.
- **Não assuma um tamanho fixo para `wchar_t`.** Como vimos, ele varia entre sistemas (2 bytes no Windows, 4 no Linux), o que compromete a portabilidade de código que dependa desse tamanho específico.
- **Cuidado com truncamento no meio de um caractere multibyte.** Cortar uma string UTF-8 em um índice de byte arbitrário (por exemplo, ao limitar um campo a "N caracteres" usando apenas `strncpy`) pode partir um caractere ao meio, produzindo bytes inválidos ao final da string resultante.
- **Padrões modernos do C** (a partir do C11) introduziram os tipos `char16_t` e `char32_t` (em `<uchar.h>`), com tamanhos fixos e bem definidos — 16 e 32 bits, respectivamente — justamente para reduzir a dependência da definição de `wchar_t`, que varia por sistema.
- **Na dúvida, prefira UTF-8** para armazenamento e transmissão de texto (arquivos, redes, bancos de dados), e trate a decodificação para *code points* individuais apenas nos pontos do programa em que isso for genuinamente necessário — em vez de converter o programa inteiro para trabalhar com `wchar_t`.

Conclusão

Começamos este capítulo com uma pergunta aparentemente ingênua — "o que é um caractere?" — e terminamos vendo que ela esconde três respostas diferentes (byte, *code*

point, grafema), uma história de mais de sessenta anos de padronização, e decisões de projeto que ainda hoje diferem entre Windows e Linux.

Nada disso muda o que você aprendeu no Capítulo 6: para a esmagadora maioria dos programas que você vai escrever durante o curso, tratar uma string como um vetor de `char` terminado em `'\0'` continua sendo o modelo correto e suficiente. O que muda é a sua compreensão do que está por trás dessa simplificação e, principalmente, o reconhecimento de quando ela deixa de ser suficiente: no momento em que seu programa precisa lidar, com correção, com texto que vai além do inglês.

Se este capítulo despertou seu interesse, os capítulos de *strings* e *ponteiros* de *Understanding and Using C Pointers* (Reese) são uma continuação natural especialmente para quem quer entender como bibliotecas de manipulação de UTF-8 são implementadas, byte a byte, em cima da mesma aritmética de ponteiros que estudamos na Parte III.

...